

Содержание

1	Операционные системы	4
2	История UNIX/Linux	4
3	Современность	4
4	Ядро *NIX	5
5	Файловая система	6
6	Права доступа к файлам	7
7	Способы задания прав	7
8	Потоки stdin(0), stdout(1), stderr(2)	7
9	Интерпретатор команд	7
10	Перенаправление потоков stdin(0), stdout(1), stderr(2)	8
11	Фильтры	9
12	Регулярные выражения	9
13	Команды	10
14	Аналоговые ЭВМ	10
15	Цифровые ЭВМ	11
16	Функциональные элементы ЭВМ	12
17	Первая ЭВМ: Калькулятор (1)	19
18	Архитектура ЭВМ, Гарвардская, Фон Неймана.	22
18.1	Фон-Неймана	22
18.2	Гарвардская	23
19	Структура БЭВМ-NG	24
20	Устройство Управления	24
21	Адресуемая память БЭВМ	25
22	АЛУ, коммутатор, блок признаков результата	25
23	Форматы команд	25
24	Адресные команды	27
25	Безадресные команды	27
26	Представление чисел: фиксированная точка	28
27	Представление беззнаковых целых чисел	29

28	Представление знаковых целых чисел	30
29	Представление знаковых чисел: дополнительный код	31
30	Перенос, Переполнение	33
31	БЭВМ: представление чисел	34
32	Представление чисел с плавающей точкой	38
33	Представление логической информации	39
34	Представление символьной и текстовой информации	40
35	Символы: ASCII	41
36	Символы: ASCII (КОИ-7Н0)КОИ-7Н1 (РУС),КОИ7Н2 (Mix)	42
37	Символы: КОИ-8	42
38	Символы: ISO8859-5 (ГОСТ-основная)	42
39	Символы: WIN1251	43
40	Символы: UNICODE, UTF-8	43
41	Big-endian и Little-endian	44
42	Представление строк	44
43	История развития ЭВМ	46
44	История развития ЭВМ в СССР/России	47
45	Канальная организация	48
46	Раздельные шины	49
47	Общие шины	50
48	Мультиплексирование шин	51
49	Мультипроцессорность:UMA - Uniform Memory Access	52
50	Мультипроцессорность: Коммутатор	53
51	Мультипроцессорность: NUMA – Non Uniform Memory Access	54
52	Современные коммерческие процессоры	55
53	CISC, RISC, VLIW	56
54	Характеристики памяти	57
55	Статическая vs Динамическая память	59
56	Адресуемая память	62

57	Адресуемая память с фиксацией строк и столбцов	64
58	Синхронная память SDRAM	65
59	Конструктивные особенности современной памяти	65
60	Память, ориентированная на записи	66
61	Память, с последовательным доступом	67
62	Структура ассоциативного запоминающего устройства	68
63	Кэш память	69
64	Пирамида памяти	70
65	Влияние промахов кэш-памяти	71
66	Сегментно-страничная виртуальная память	71
67	MMU и TLB	73
68	История сети Internet	74
69	Понятие сети ЭВМ	75
70	Сообщение, пакет	76
71	Модель взаимодействия открытых систем (OSI)	76
72	Модель TCP/IP	77
73	Уровень передающей среды	78
74	Канальный уровень Ethernet	78
75	Сетевой уровень IP	79
76	Сетевой уровень IP: маршрутизация	79
77	DHCP	80
78	Сервис имен DNS и другие	80
79	Транспортный уровень	80
80	Прикладной уровень	80

1 Операционные системы

- Исторически предназначены для замены работы оператора компьютерной системы
- Условно делятся на
 - Пользовательские - интерактивность
 - Серверные - обеспечивает большой поток задач
 - Встроенные ОС
- Windows, Linux/Unix, Android/iOS, VXWorks/EmbeddedLinux/DOS, Гипервизоры
- Обычно включают ядро с подсистемами управления памятью и процессами и драйверы устройств

2 История UNIX/Linux

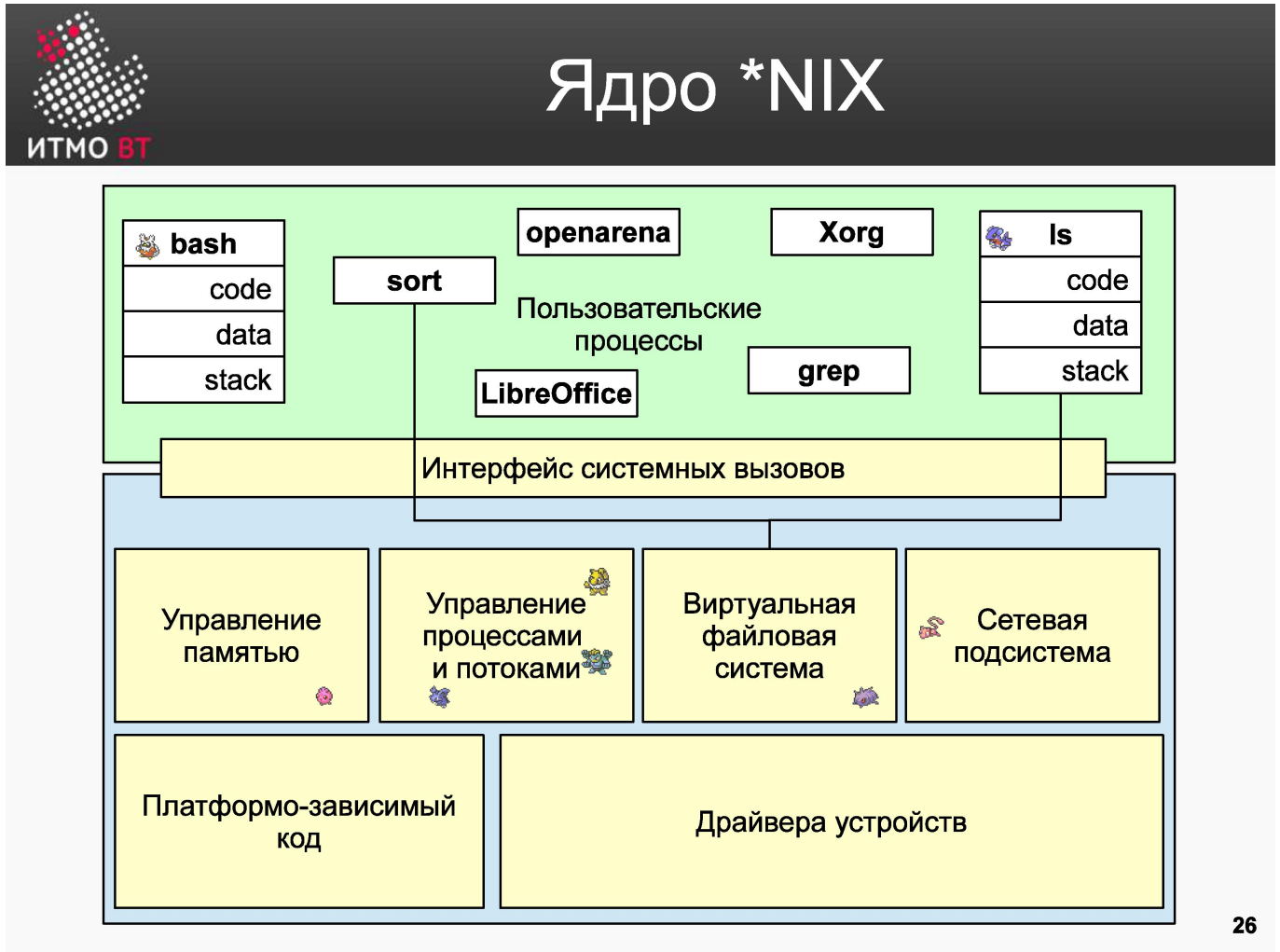
3 основные ветки

- Потомки BSD - развитая сетевая подсистема
- Потомки System-V - уровень Enterprise. Работают под управлением машин, которые никогда не останавливаются
- Linux (Unix подобная, библиотеки/интерфейсы вызова остались похожими)

3 Современность

- Коммерческие - System V
 - Solaris, HIX
- BSD
 - Mac OS X
 - NetBSD, FreeBSD, OpenBSD
- Linux
 - RedHat, Ubuntu, SUSE
 - Fedora, Debian, OpenSUSE, ArchLinux
 - Gentoo

4 Ядро *NIX



26

Ядро - набор подсистем, которые управляют оборудованием и программами пользователя.

Основные подсистемы:

- управление памятью
- управление процессами и потоками (отвечает за выполнение нескольких программ одновременно на одной ОС) виртуальная файловая система (обмен данными организован с помощью файлов, с точки зрения ОС в UNIX все является файлами)
- сетевая подсистема
- драйвера устройств
- платформено-зависимый код (драйвера, которые поддерживают функционирование системы на различных аппаратных архитектурах)

Взаимодействие программ пользователя с ядром происходит с помощью интерфейса системных вызовов.

С точки зрения пользователя любая исполняемая программа представляет собой адресное пространство, которое разбито логически и физически на несколько сегментов.

Например:

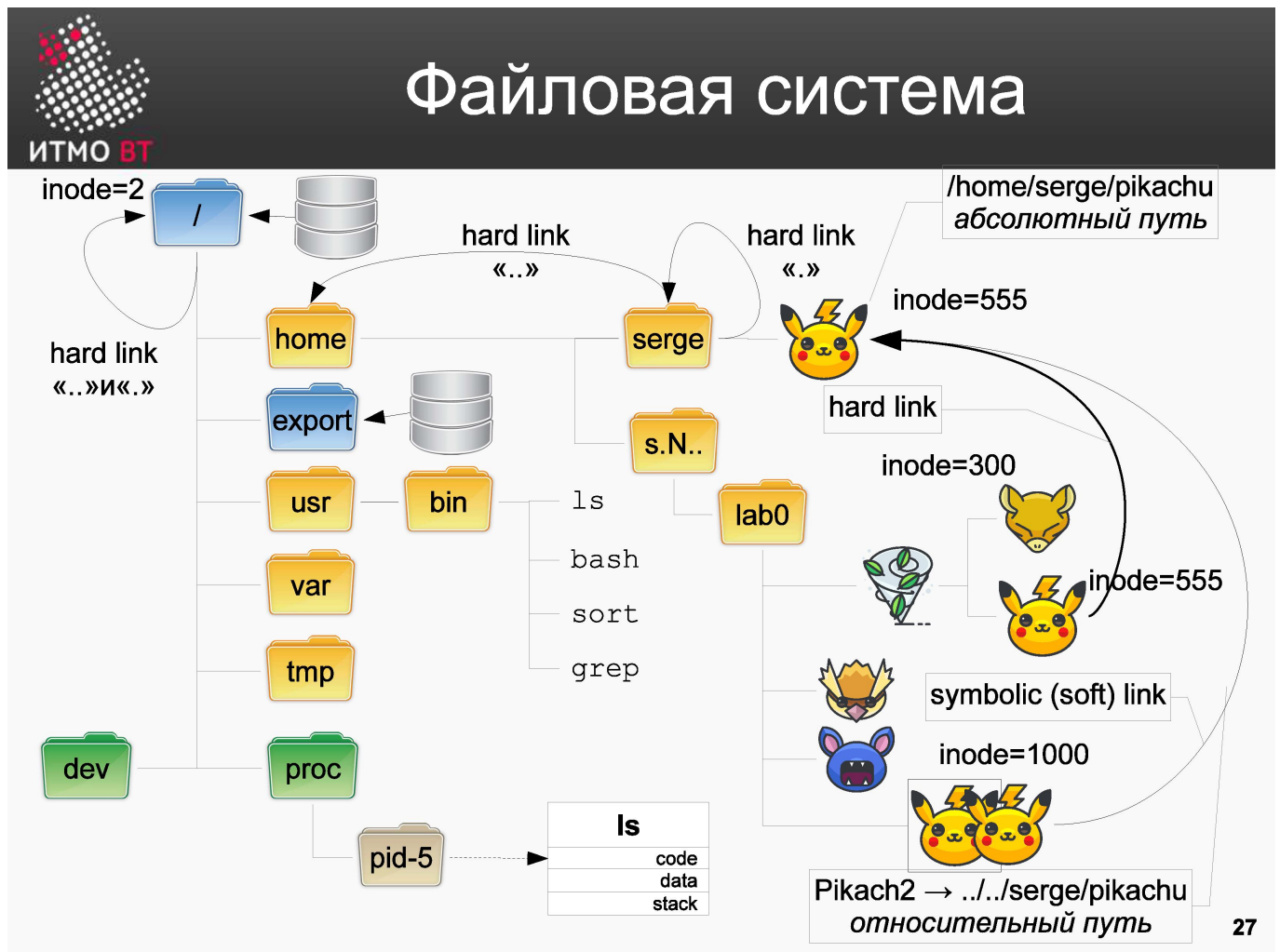
- сегмент code, где выполняется собственно сама программа
- сегмент data, где находятся необходимые для ее работы данные

- сегмент `stack`, для хранения адресов возврата и передачи аргументов к подпрограммам.
- `heap`(или куча)
- сегменты разделяемых библиотек
- `memory-mapped` файлы, т. е. файлы непосредственно отображенные в адресное пространство процесса и другие

Все эти сегменты вместе составляют образ процесса, который работает внутри операционной системы и взаимодействует с ней.

Пользовательские процессы могут получить сервисы ядра только через этот интерфейс.

5 Файловая система



Файловая система (ФС) - часть операционной системы, представляющая собой совокупность организованных наборов данных, хранящихся на внешних запоминающих устройствах и программных средств, гарантирующих именованный доступ к этим данным и их защиту. Данные называются файлами, их имена - именами файлов.

Файловые системы можно классифицировать по степени персонификации доступа к содержимому файлов, соответственно могут быть:

- однопользовательские файловые системы
- многопользовательские файловые системы
- Все ресурсы представлены в виде файлов

6 Права доступа к файлам

Права доступа разбиты на несколько групп и определяют права для разных категорий пользователей: Все типы прав содержат три значения (установленное, если право дано, и неустановленное, если право отсутствует): право на чтение (r), право на запись (w), право на исполнение (x). Существующие типы файлов: файл, директория и символическая ссылка. Unix каждый из этих типов рассматривает как файл. Право на исполнение для директории означает, что в директорию можно “зайти” (например, с помощью cd).

7 Способы задания прав

- Буквенные обозначения (символьный код) u - владелец; g - группа-владелец; o - остальные пользователи; a - все пользователи (u + g + o). “+” - добавить права; “-” - удалить права; “=” - задать права (удалить предыдущие и задать новые). Право на чтение (r); на запись (w); на выполнение (x). Пример: `chmod u=rw, g-r, o+x file` Задали права для владельца: чтение и запись Для группы-владелец: отняли право на чтение Для остальных пользователей: добавили право на исполнение.
- Числовой код (указание прав в виде восьмеричного числа). Пример: `chmod 335 graveler0/kingdra` Первая цифра - права доступа для владельца; вторая - для группы-владелец; третья - для остальных пользователей. Право на чтение (r) = 4; на запись (w) = 2; на выполнение (x) = 1. Значит, права владельца: w + x = 2 + 1 (запись и выполнение). Права группы-владелец: w + x = 2 + 1 (запись и выполнение). Права остальных: r + x = 4 + 1 (чтение и выполнение).

8 Поток stdin(0), stdout(1), stderr(2)

- `stdin(0)` - поток стандартного ввода; используется для получения данных от пользователя или других источников; по умолчанию связан с терминалом, что позволяет программе считывать вводимые пользователем данные с клавиатуры.
- `stdout(1)` - поток стандартного вывода; предназначен для вывода данных из программы; по умолчанию также связан с терминалом.
- `stderr(2)` - поток вывода диагностических сообщений об ошибках; используется для вывода сообщений об ошибках и предупреждений; по умолчанию связан с терминалом.

По умолчанию все потоки связаны с виртуальным терминалом, откуда была запущена программа. Наличие двух потоков вывода необходимо для того, чтобы разделить вывод “полезных данных” и ошибок работы программы и дальнейшего перенаправления их, например, в файлы или другим командам.

9 Интерпретатор команд

Всеми командами (программами и приложениями) управляет интерпретатор команд - shell. Эта программная оболочка позволяет организовывать выполнение команд, задавать им переменные окружения, связывать потоки ввода-вывода у различных команд между собой, а также анализировать коды возврата у команд и использовать простейший язык для написания новых исполняемых команд - скриптов.

Различных интерпретаторов существует достаточно большое количество. Исторически shell был командной оболочкой, которая использовалась пользователями до появления первого графического интерфейса. В настоящее время в серверных ОС shell используется для доступа и настройки систем

удаленными от сервера администратора. Наиболее популярной оболочкой для Unix является bash (не во всем ОС установлен по умолчанию).

При запуске пользовательской программы bash сначала связывает её потоки с терминалом, из которого он был запущен. Затем, если указаны специальные символы перенаправления ввода-вывода (как раз потоки `stdin(0)`, `stdout(1)`, `stderr(2)`), bash подключит к потокам программы необходимые файлы или свяжет команды между собой, что часто используется для организации фильтрации потоков.

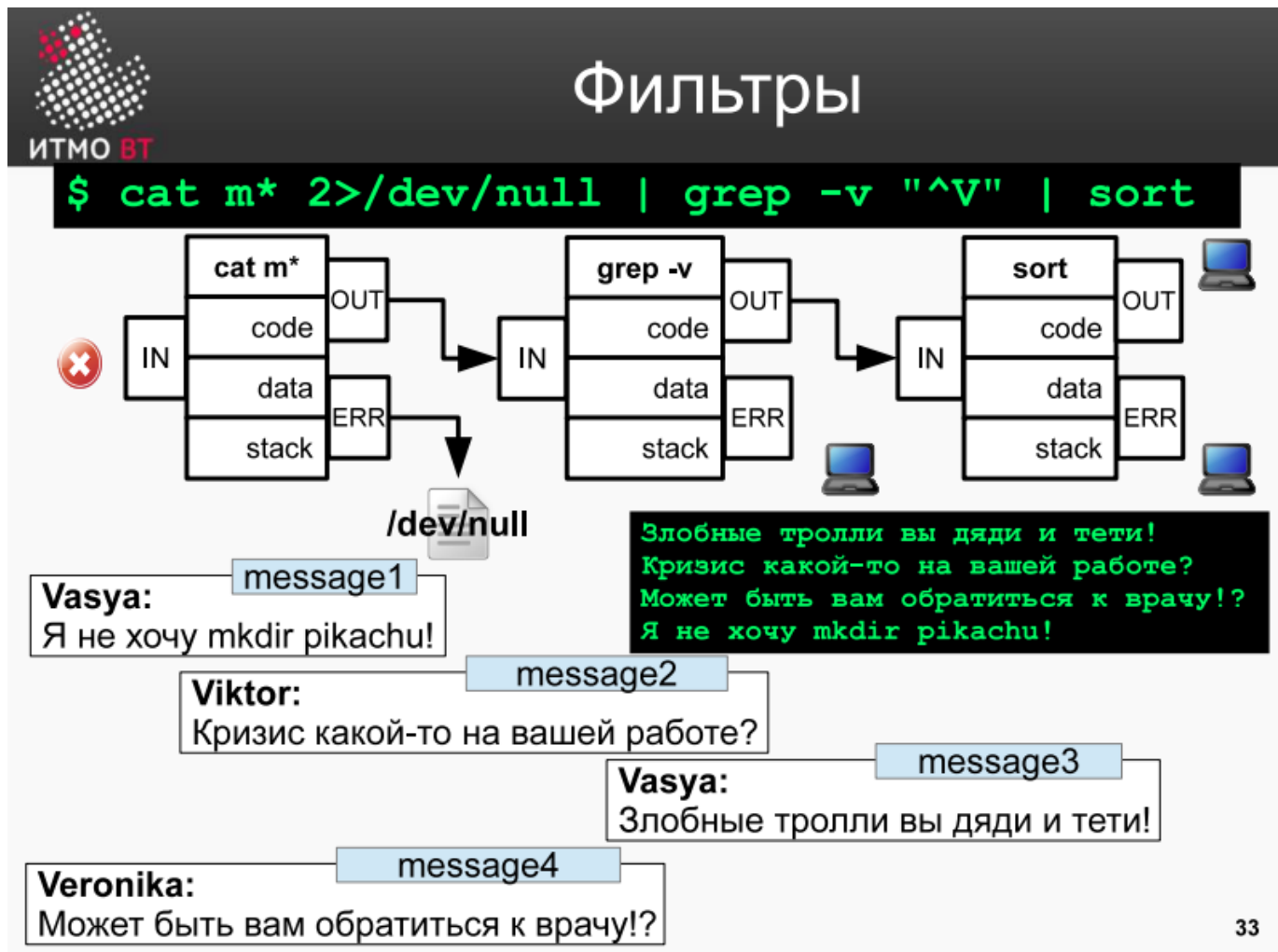
10 Перенаправление потоков `stdin(0)`, `stdout(1)`, `stderr(2)`

Специальные символы shell для перенаправления потоков ввода-вывода:

- `> file` - перенаправить `stdout` в `file`
- `» file` - добавить `stdout` в `file`
- `2> file` - перенаправить `stderr` в `file`
- `2» file` - добавить `stderr` в `file`
- `< file` - взять `stdin` из `file`
- `« EOF` - записать `stdin` из терминала до символов “EOF”

Символ “|” (конвейер) необходим для связывания стандартного вывода команды слева от конвейера со стандартным вводом команды справа (как бы передает данные от одной команды к другой).

11 Фильтры



“Фильтры - это программы, которые позволяют просто для пользователя фильтровать информацию.”

© Клименков

Фильтры позволяют сильно упрощать взаимодействие с компьютером, если их освоить.

Основные - `grep` и `sort`.

grep - по умолчанию выделяет строки, соответствующие регулярным выражениям - удобный инструмент для поиска необходимых строк или подстрок. **sort** - по умолчанию сортирует по алфавиту (обладает множеством других ключей, которые позволяют сортировать, например, по числу -n, по месяцу -M и т.д.) - удобная команда для сортировки входных данных.

Очень часто используются вместе с конвейером `|`, например, `cat m* | grep -v ^V | sort`

Здесь мы выведем содержимое всех файлов, удовлетворяющих маске `m*`, перенаправим вывод команды `cat` на `grep`, `grep` оставит только строки, НЕ (флаг `-v`) начинающиеся на `V`, `sort`, очевидно, отсортирует по алфавиту.

12 Регулярные выражения

Регулярные выражения - формальный язык, используемый в компьютерных программах, работающих с текстом, для поиска и осуществления манипуляций с подстроками в тексте, основанный на использовании метасимволов

Тесно связаны с фильтрами.

Например:

Некоторый символ - соответствие самому себе

- ^ - начало строки
- \$ - конец строки
- .

Например, “т.\$” - поиск строки, содержащей символ “т”, любой символ и еще один любой символ на конце.

Команда `grep` как раз использует `regex`:

`grep “т.$”` - поиск строк, удовлетворяющих данному регулярному выражению

Также, например, у `grep` существует ключ `-E` - используется для использования `ExtendedRegex` - позволяет, например, использовать `capture-group`. Очень удобно для поиска повторяющихся символов или целых групп:

13 Команды

Команда	Назначение и синтаксис
<code>mkdir</code>	<code>mkdir [-m mode] [-p] dir...</code>
<code>echo</code>	<code>echo [string]...</code>
<code>cat</code>	<code>cat [-n] [file...] [-]</code>
<code>touch</code>	<code>touch [-am]... file...</code>
<code>ls</code>	<code>ls [options] [file/dir]...</code>
<code>pwd</code>	<code>pwd</code>
<code>cd</code>	<code>cd [argument]</code>
<code>more</code>	<code>more [file...]</code>
<code>cp</code>	<code>cp [options] SOURCE ... DEST</code>
<code>rm</code>	<code>rm [options] [file/dir]</code>
<code>rmdir</code>	<code>rmdir [dir]</code>
<code>mv</code>	<code>mv [-fi] SOURCE ... DEST</code>
<code>head</code>	<code>head [-num] [file...]</code>
<code>tail</code>	<code>tail [-/+num] [-bcl] [file...]</code>
<code>sort</code>	<code>sort [-unr] [-k num] [file...]</code>
<code>grep</code>	<code>grep [-v] regexp [file...]</code>
<code>wc</code>	<code>wc [-c -m] [-lw] [file...]</code>

14 Аналоговые ЭВМ

Имеют высокое быстродействие и имеют меньшую погрешность вычислений, но подходят только для определённого рода задач.

Например, дально/глубиномер - имеет высокое быстродействие, но ни для чего больше не может использоваться. Также информацию нельзя дискретизировать в аналоговых ЭВМ, сигнал подаётся непрерывно, а получаемое значение не конечно (как это было бы, например, в ЭВМ, где сигнал либо есть, либо его нет).

Помимо того, что используются только для специализированных задач, зачастую имеют достаточно большие габариты и иногда немало движущихся частей (например, аналоговый калькулятор синуса или косинуса).

Величины, которые считает аналоговая ЭВМ, изначально не привычны и неудобны для человека, поэтому для их преобразования в “человеческий” вид используются сложные технические решения, поскольку, как оговаривалось ранее, сигнал не дискретен и переводить его приходится по различным уровням сигнала, что совсем неудобно. Вследствие этого пришли к цифровым ЭВМ, где сигнал либо присутствует, либо отсутствует, и измерять его очень легко.

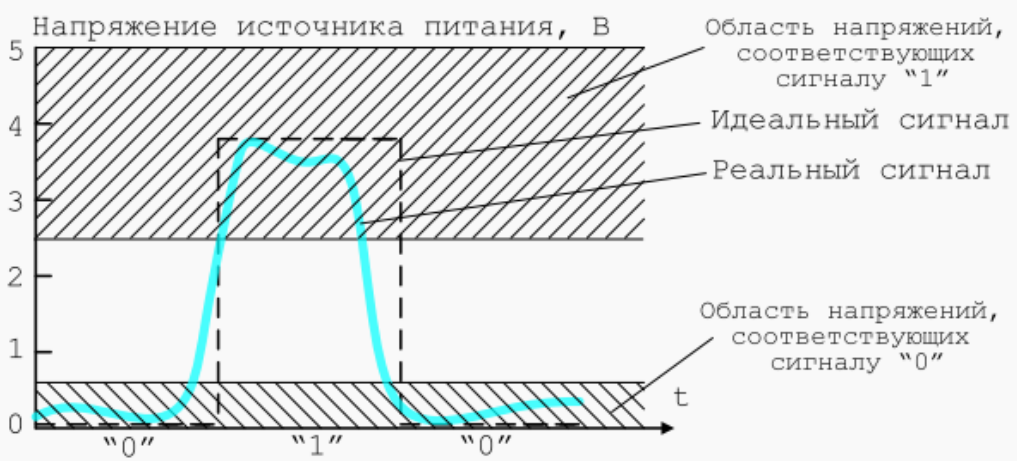
Тем не менее, аналоговые ЭВМ можно успешно использовать в специализированных задачах, где они отлично справляются со своей работой.

15 Цифровые ЭВМ



Цифровые ЭВМ

- Представления информации с помощью только двух дискретных величин — 0 и 1



39

Зачастую на схемах сигнал рисуется как идеальный прямоугольник, в реальности он таким не является. Если сигнал находится между областями (т.е. сильно искажён), где определены сигналы 1 и 0, то будут появляться ошибки. Такое происходит часто, но специальные аппаратные решения эти ошибки минимизируют.

Вскоре люди научились создавать подобные машины более простыми и ёмкими, чем аналоговые, вследствие чего они быстро завоевали рынок.

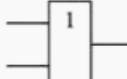
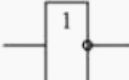
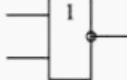
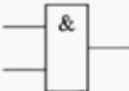
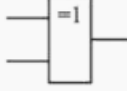
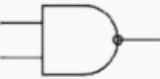
16 Функциональные элементы ЭВМ

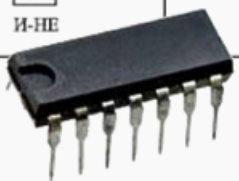
Базой любой вычислительной машины являются логические элементы:



Функциональные элементы ЭВМ

• Логические элементы

ГОСТ	ANSI	ГОСТ	ANSI
 Буфер	 BUF	 ИЛИ	 OR
 Инвертор	 INV	 ИЛИ-НЕ	 NOR
 И	 AND	 Исключающее ИЛИ	 XOR
 И-НЕ	 NAND	 Исключающее ИЛИ-НЕ	 XNOR

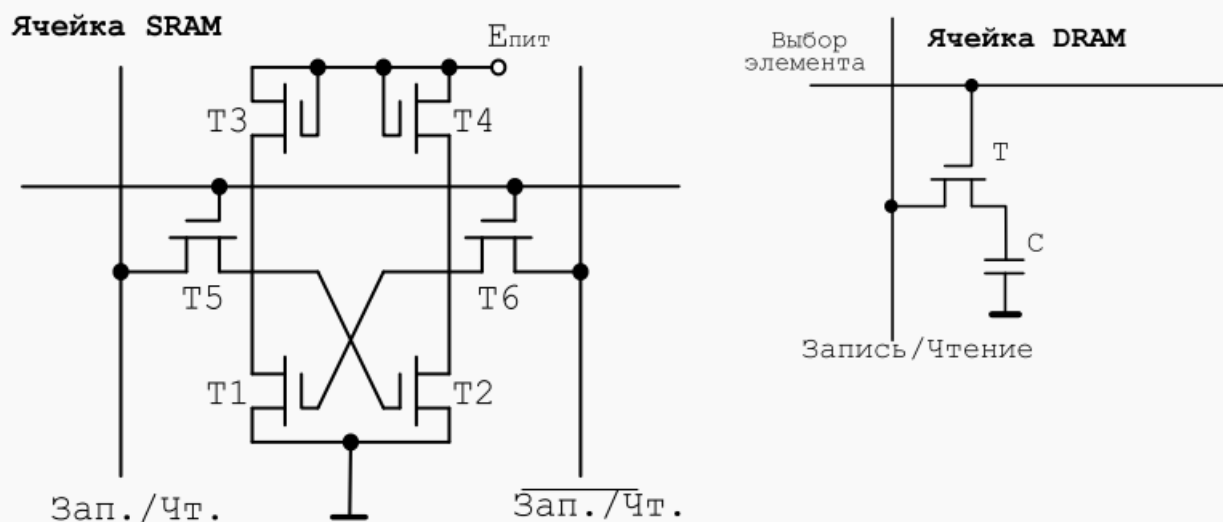


X1	X2	Y	$\bar{Y}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

X1	X2	Y	$\bar{Y}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

X1	X2	Y	$\bar{Y}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

• Элементы хранения (DRAM/SRAM)

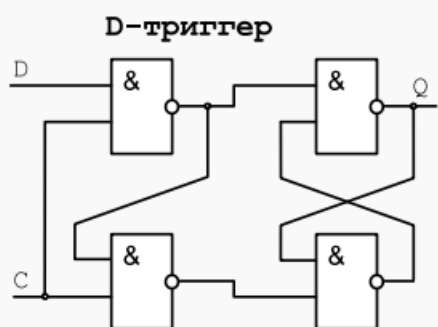


41

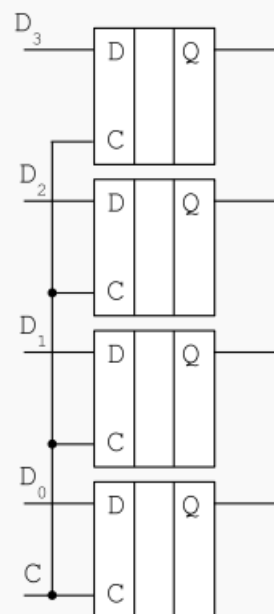
DRAM (можно даже схему зарисовать) - динамическая память - дешёвая, простая, схема маленькая, что позволяет достигнуть большего объёма на одной площади по сравнению с SRAM, но медленнее, чем SRAM. Используется в качестве оперативной памяти (ну те самые плашки)

SRAM - статическая память - дорогая, сложная, большая схема, но быстрее, чем DRAM, также не приходится постоянно обновлять память. Используется в регистрах. Одна такая ячейка зовётся триггером. Память в процессоре построена на подобных статических элементах.

• Элементы хранения (триггеры, регистры)



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	0	1	0	0	0	0	0	1	0	1	1



42

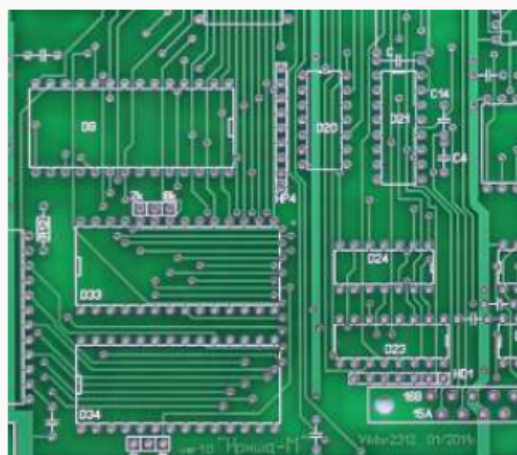
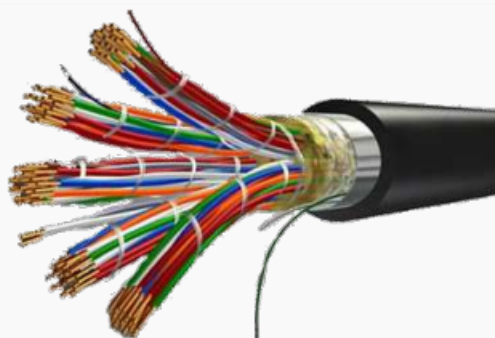
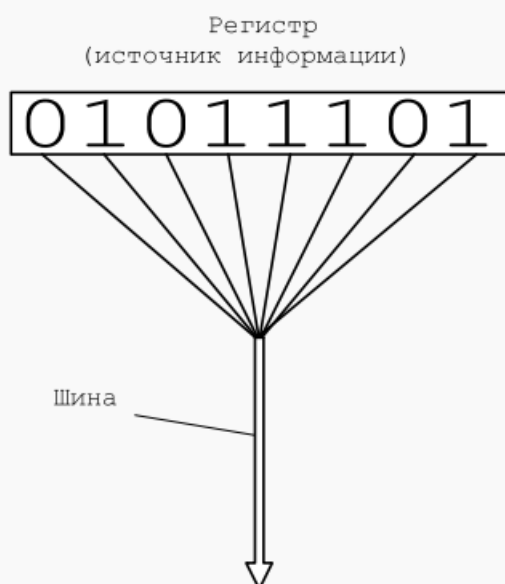
Регистр - совокупность единичных триггеров, у которых объединён управляющий вход с в один.

D - вход

Q - выход

C - управляющий сигнал

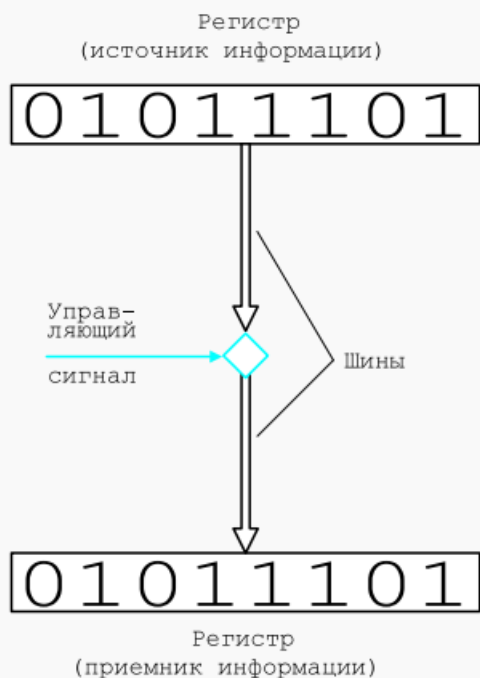
- Провода, шины



43

Элемент, который объединяет в себе несколько проводов обычно называют шиной, по которым передаётся информация схожего значения.

• Вентили



Вентиль (И)		
Упр.	Инф.	Вых.
0	0	0
0	1	0
1	0	0
1	1	1

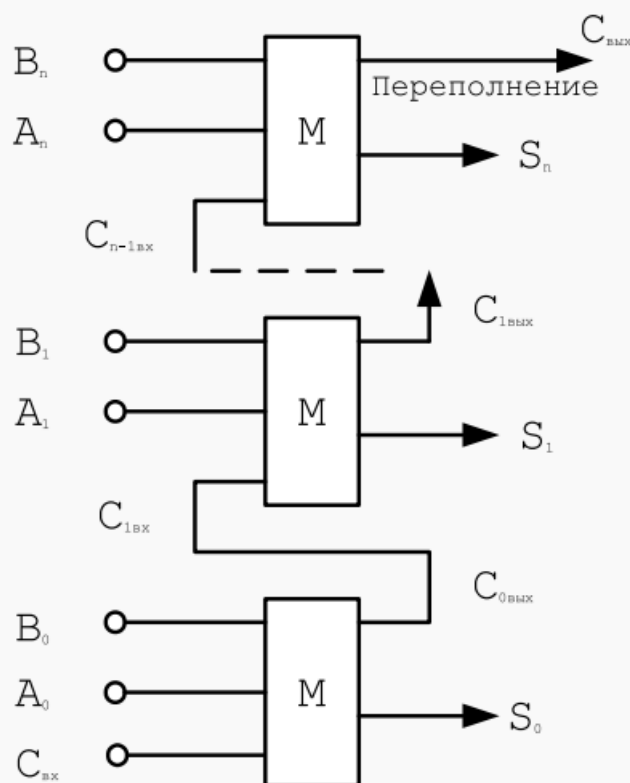


44

Вентили используются для управления передачей информации, обычно представляется схемой логического “и”. Т.е. он как бы разрешает пропуск информации.

• Сумматоры (входят в АЛУ)

SUM				
Свх	Ai	Bi	Свых	Si
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



45

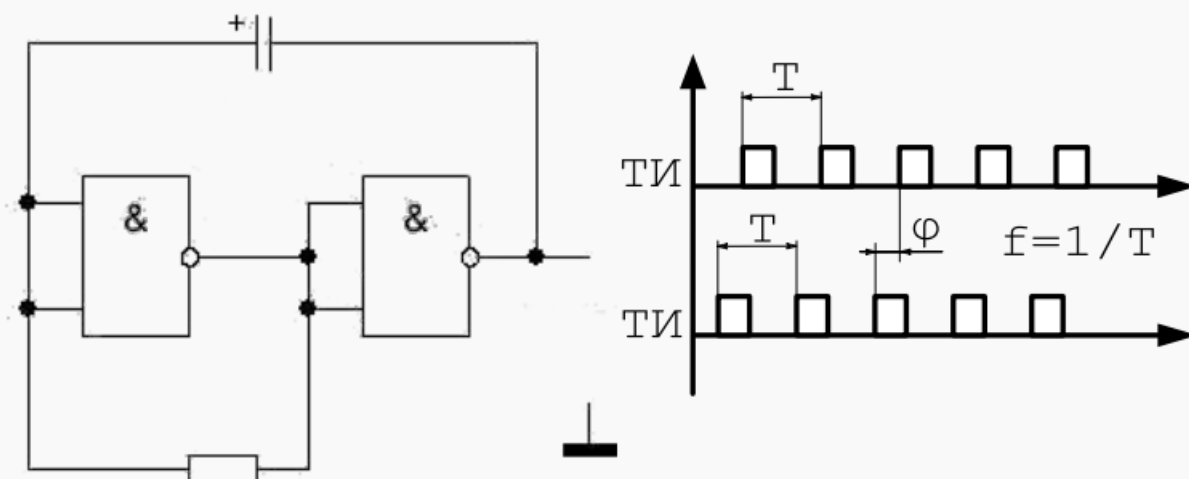
Сумматор имеет три входа и два выхода:

A - 1 число B - 2 число C вх- бит переноса

C вых - бит переноса S - результат сложения

Целью его является сложение (и вычитание при помощи доп кода) чисел. Он входит в состав АЛУ. Также скорость сумматора зависит от разрядности чисел, которые он складывает. сложение однобитных чисел ровно в 16 раз быстрее сложения 16 битных чисел. Если бы у нас была необходимость умножать 16 битные числа, но необходимо было бы иметь 32 бита для хранения результата.

- Тактовые генераторы



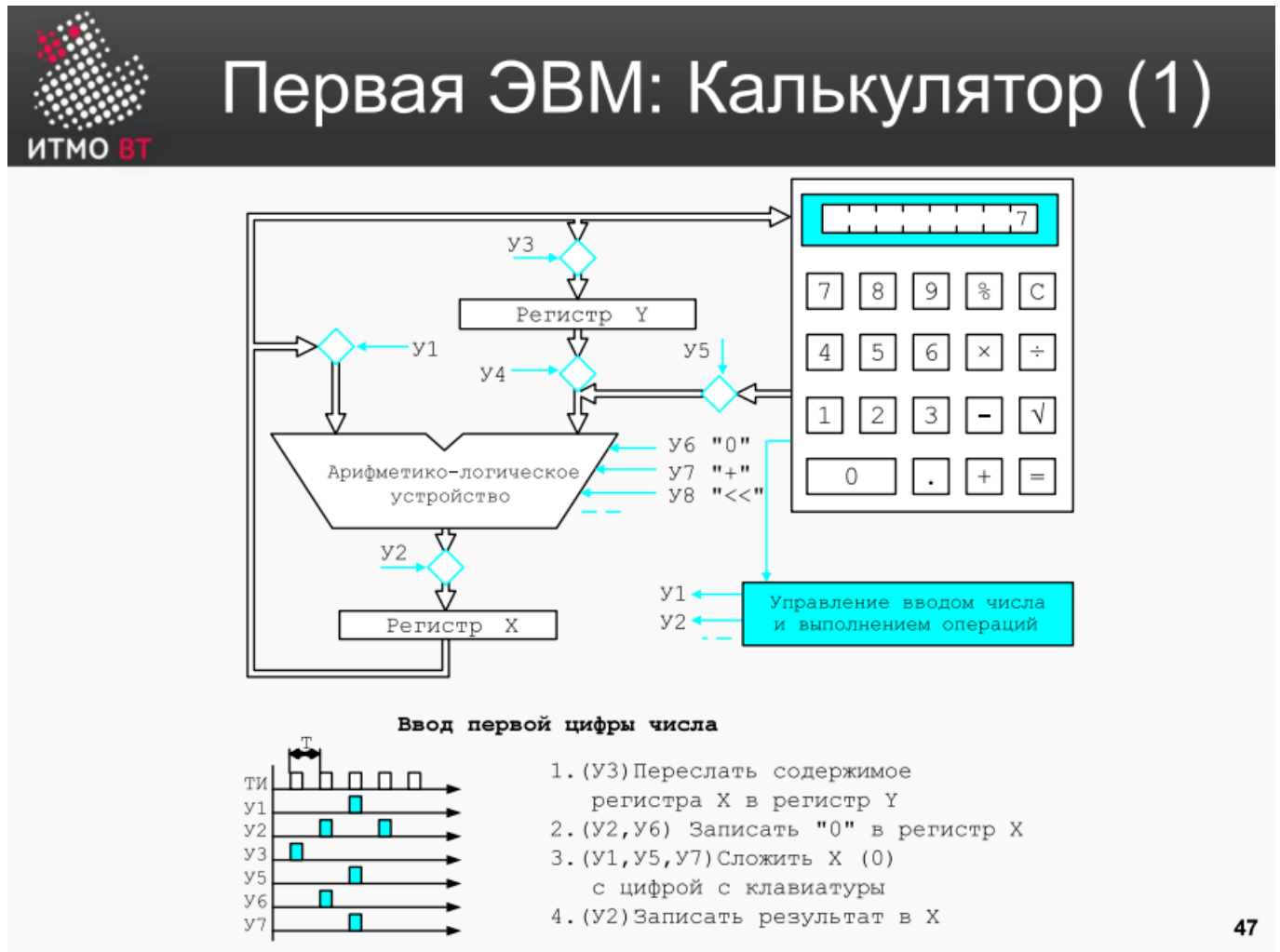
46

“Сердце” компьютера.

Тактовый генератор порождает импульсы с заданной частотой и некоторыми другими параметрами. Частота импульсов должна быть постоянной, так как все операции должны уложиться “внутри” нашего тактового импульса, частота тактового генератора = частота процессора. И все элементы обычно работают от одного тактового генератора, так как между двумя импульсами абсолютно одинаковых тактовых генераторов может быть сдвиг.

Просто так повысить частоту нельзя, поскольку тепловыделение будет увеличиваться **ОЧЕНЬ** быстро. Также, если процессор перегревается, он может сбрасывать частоту, этот процесс называется пропуском тактов или тротлингом.

17 Первая ЭВМ: Калькулятор (1)



Из деталей, числа представлены в двоично-десятичном виде. Т.е. каждый десятичный разряд в числе представлен 4 битами.

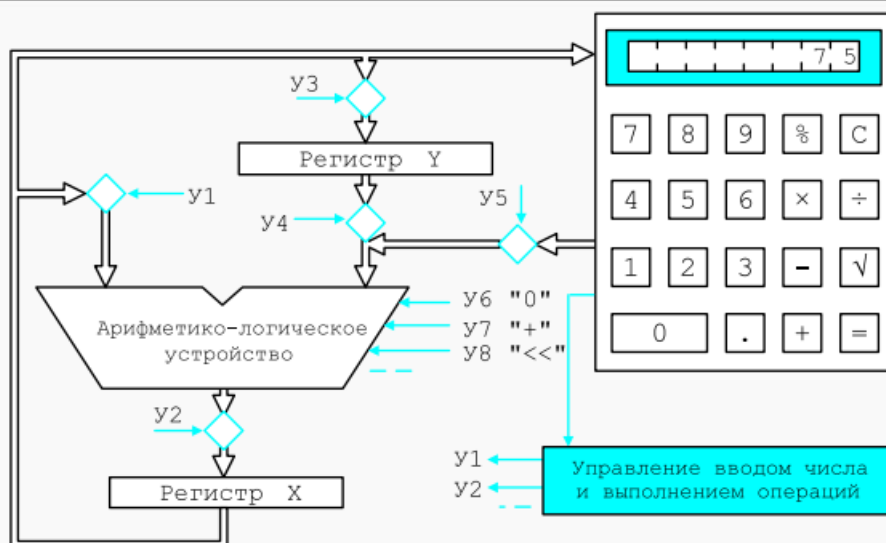
Y5 пускает на ввод код кнопки, которую вы нажали, оно приходит в АЛУ. Регистр X хранит ваше число, которое вы видите, Y - хранит второе число, с которым вы проводите операцию. Y2 позволяет записать в регистр X, этот самый регистр напрямую связан с экраном. Y4 пускает на правый вход алу число из регистра Y, Y3 позволяет запись в регистр Y, Y1 пускает содержимое регистра X на левый вход АЛУ.

По сути вам просто следует описать, а в идеале зарисовать (уж я без понятия как будет это задание оцениваться) схему, представленную выше и рассказать, как происходит сложение двух чисел. Т.е. то, что ниже схемки калькулятора.

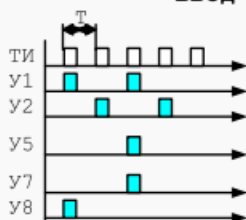
В целом, звучит логично:

записать X в Y (открыть Y3) занулить X (открыть Y6, Y2) сложить X и Y (передать Y на левый вход АЛУ, передать число на правый вход АЛУ, сложить (открыть Y7)) записать в X (открыть Y2)

Первая ЭВМ: Калькулятор (2)

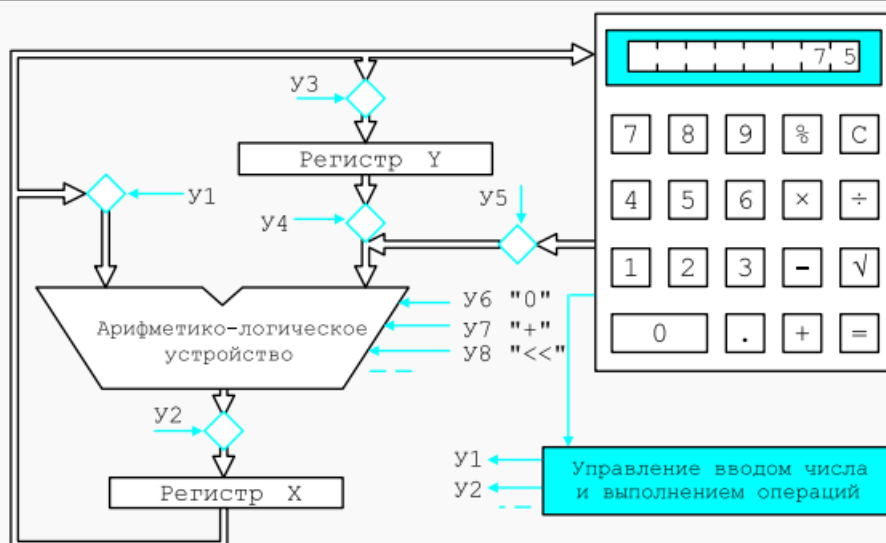


Ввод второй (и последующих) цифр числа

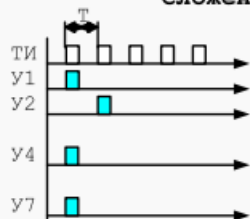


1. (y1, y8) Сдвинуть содержимое регистра X на 1 разряд (*10)
2. (y2) Записать результат в регистр X
3. (y1, y5, y7) Сложить X с цифрой с клавиатуры
4. (y2) Записать результат в X

Первая ЭВМ: Калькулятор (3)

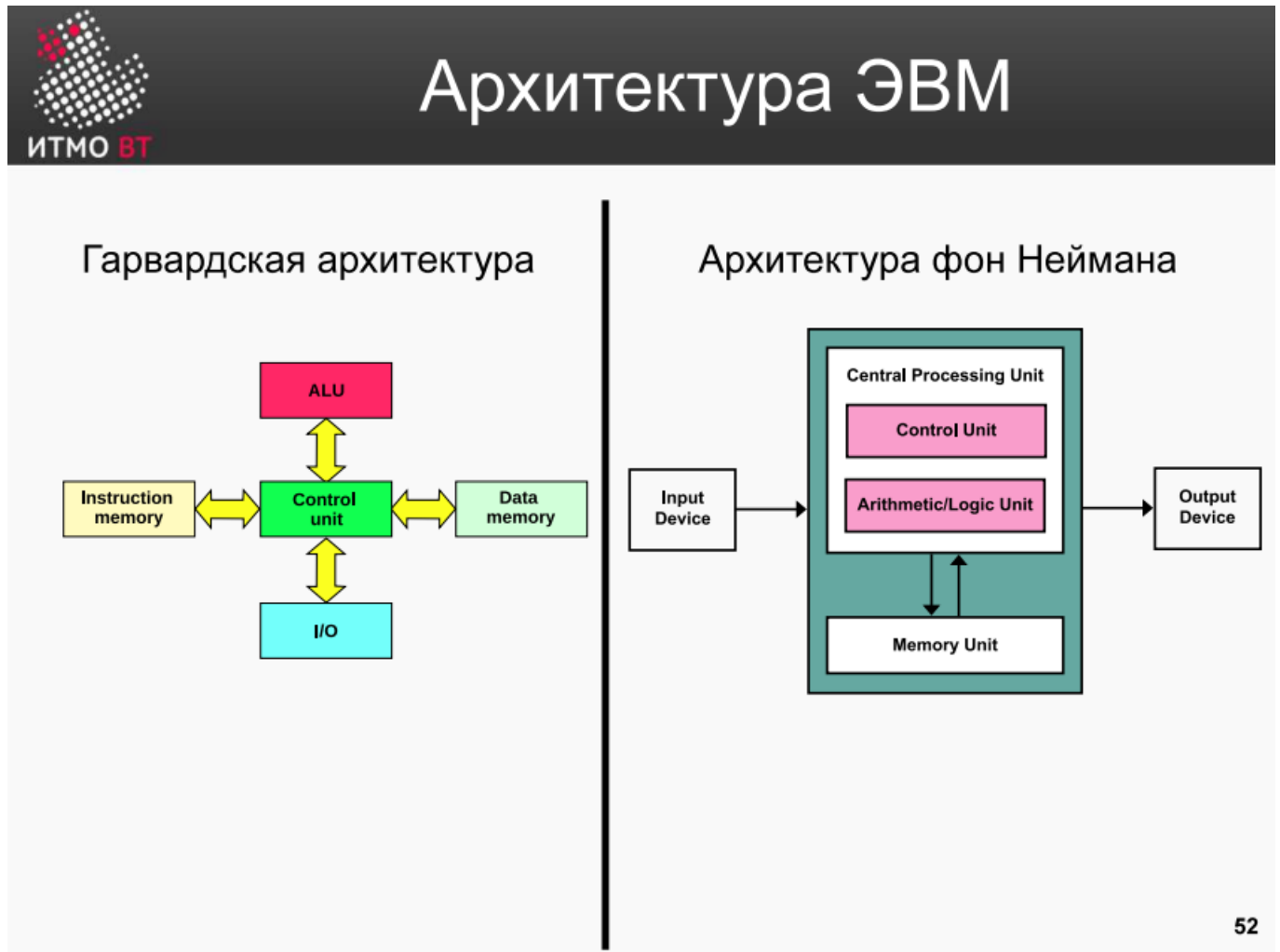


Сложение регистра X и регистра Y



1. (Y1, Y4, Y7) Сложить содержимое регистра X и регистра Y
2. (Y2) Записать результат в регистр X

18 Архитектура ЭВМ, Гарвардская, Фон Неймана.



18.1 Фон-Неймана

:

- Общая память инструкций и данных
 - Программа (инструкции) и данные хранятся в одной и той же памяти.
 - Один и тот же адресный и управляющий механизм используется для доступа к инструкциям и данным.
- Общая шина инструкций и данных - Данные и инструкции передаются через одну шину.

Плюсы:

- Простота конструкции
- Универсальность: подходит как для программируемых вычислений, так и для хранения данных.
- Дешевле в реализации по сравнению с Гарвардской архитектурой.

Минусы:

- Узкое место (бутылочный горлышко): из-за использования одной шины возникает задержка при одновременном обращении к данным и инструкциям.
- Менее эффективен в задачах, требующих высокой пропускной способности памяти.

18.2 Гарвардская

:

- Раздельная память для инструкций и данных
 - Инструкции и данные хранятся в отдельных блоках памяти.
 - Для каждой памяти используется собственный адресный и управляющий механизм.
- Раздельные шины данных
 - Процессор имеет две отдельные шины для доступа к инструкциям и данным.
 - Это позволяет одновременно загружать инструкции и обрабатывать данные, что значительно увеличивает производительность.
- Высокая производительность
 - Разделение памяти устраняет ”бутылочное горлышко” характерное для архитектуры фон Неймана.
 - Часто используется в специализированных устройствах (например, микроконтроллерах, DSP).

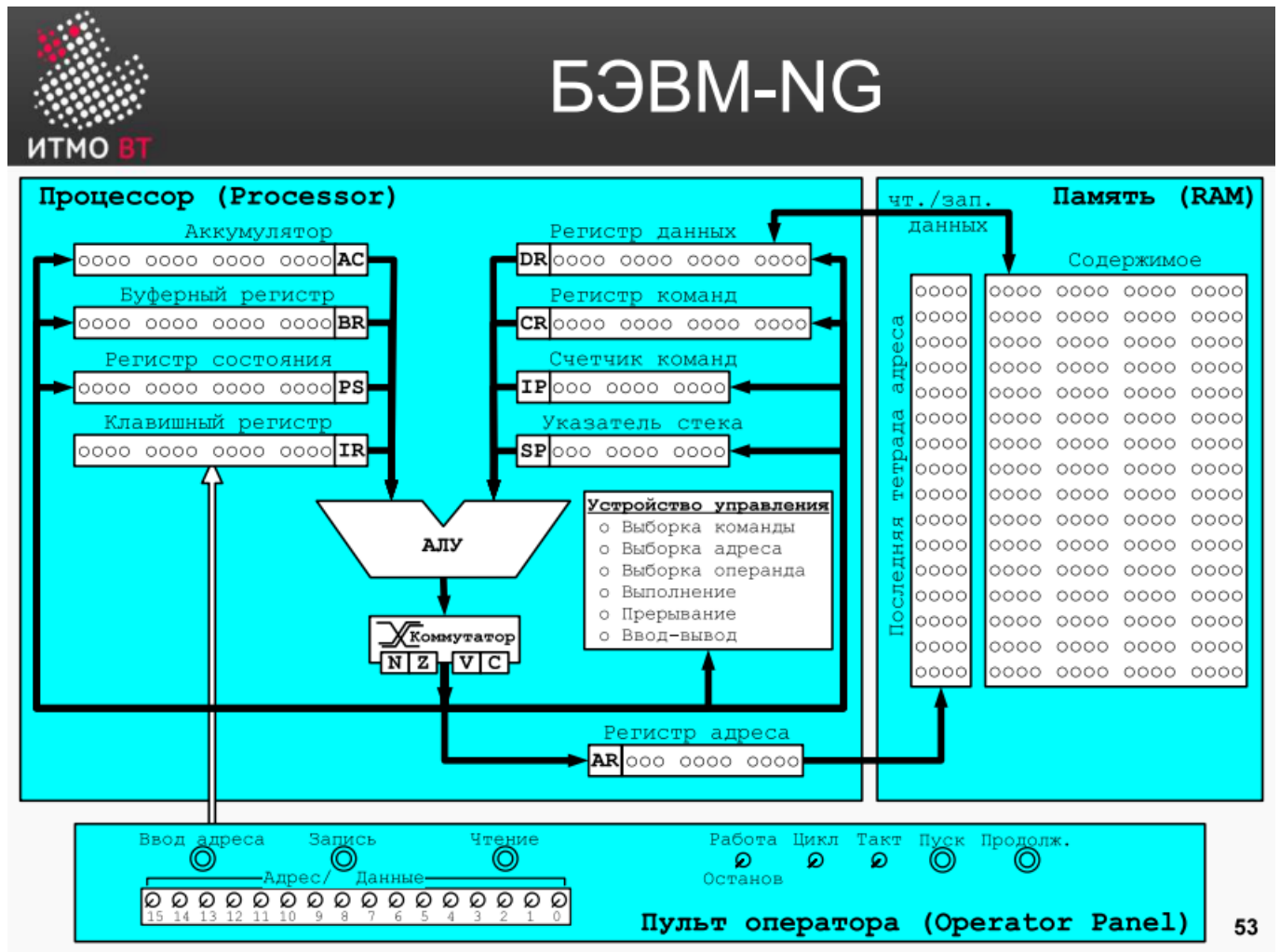
Плюсы:

- Возможность одновременного доступа к данным и инструкциям.
- Более высокая пропускная способность памяти.
- Улучшенная производительность в задачах, связанных с обработкой потоков данных (например, цифровая обработка сигналов).

Минусы:

- Более сложная и дорогая реализация из-за необходимости раздельной памяти и шин.
- Меньшая универсальность: может быть менее удобной для общего программирования.

19 Структура БЭВМ-NG



20 Устройство Управления

Цикл команды:

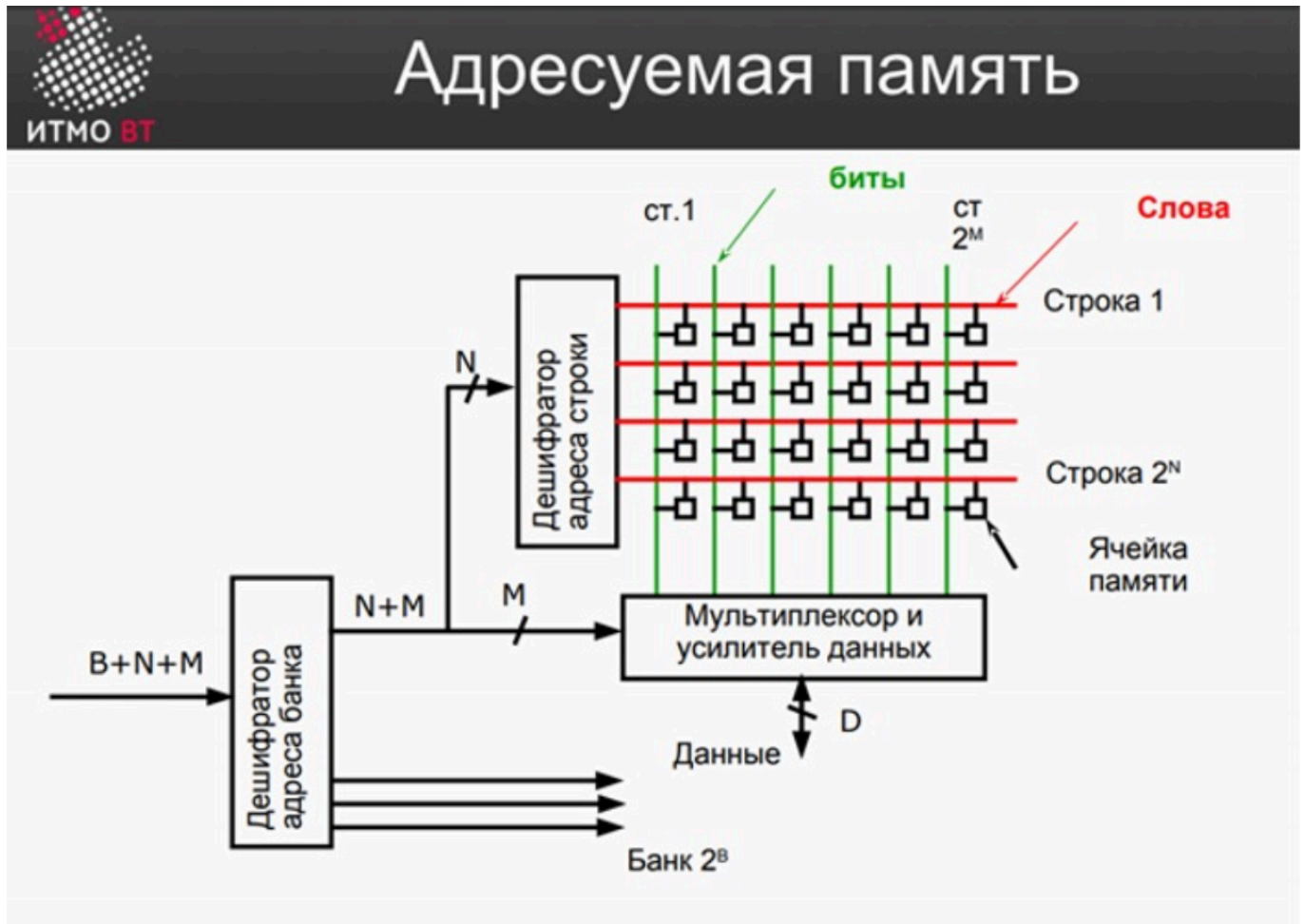
1. Цикл выборки команды: Необходим для извлечения из памяти команды, которая будет выполняться. Всегда происходят следующие действия: $IP \rightarrow BR$, $AR \rightarrow BR + 1 \rightarrow IP$, $MEM(AR) \rightarrow DR$, $DR \rightarrow CR$
2. Цикл выборки адреса - Используются для команд с относительной адресацией (следующий сем)
3. Цикл выборки операнда Выбирает данные из памяти по выбранному адресу
4. Execution - Зависит от команды
5. Interruption - Прерывание (тоже следующий сем)

Цикл пультовых операций:

- Ввод адреса
- Чтение
- Запись
- Пуск

Необходим для работы с внешними устройствами

21 Адресуемая память БЭВМ



22 АЛУ, коммутатор, блок признаков результата

Арифметико-логическое устройство (АЛУ), предназначенное для выполнения операций над содержимым регистров X и Y (сложение, умножение, вычитание и деление) или только регистра X (извлечение квадратного корня), а также ряда служебных операций (например, сдвига старших цифр числа при вводе его новой цифры);

Коммутатор — это элемент процессора или вычислительной системы, который управляет передачей данных между различными функциональными блоками, такими как регистры, арифметико-логическое устройство (АЛУ), память, и устройства ввода-вывода.

Блок признаков результата — это часть процессора, которая устанавливает специальные флаги (признаки) в зависимости от результата выполнения операции в АЛУ. Эти флаги хранятся в регистре состояния

23 Форматы команд

- АС - аккумулятор
- PS - регистр состояния

Содержит следующие биты по порядку с нулевого:

- C: перенос
- V: переполнение

- Z: ноль
- N: знак
- 0: просто ноль; используется для организации безусловных переходов в МПУ
- EI: разрешение прерываний
- IRQ: требование прерывания
- W: Состояние тумблеров РАБОТА/ОСТАНОВ
- P: тактовый генератор
- IP - Instruction Pointer - счетчик команд - регистр, хранящий адрес следующей исполняемой команды. Имеет 11 разрядов
- AR - Address Register
 - Регистр адреса
 - Служит для организации обращений к ячейкам памяти
 - Содержит в себе адрес ячейки, к которой хочет обратиться процессор
- DR - Data register
 - Регистр данных
 - Если адресная – то хранит значение ячейки к которой обращаемся
 - Если безадресная – код команды(может использоваться для хранения промежуточных значений команды)
- CR - Command Register
 - Регистр команд
 - Хранит код выполняемой в данный момент команды для поэтапного декодирования и выполнения требуемых операций
- BR - Buffer Register
 - Буферный регистр
 - чаще всего содержит адрес выполняемой программы, реже промежуточные данные для некоторых команд
- SP - Stack Pointer
 - Указатель стека
 - Как и IP имеет 11 разрядов
 - Всегда указывает на вершину стека
 - Стек - особый участок памяти
 - Предназначен для хранения:
 - * Адресов возврата;
 - * Параметров подпрограмм;
 - * Прерываний
- IR - Input Register
 - Клавишный регистр

- - Имеет 16 разрядов
- Лежит в составе пульта оператора ЭВМ
- Предназначен для:
- Ввода программы;
- Кодов программы;
- Данных

24 Адресные команды

... с прямой абсолютной адресацией

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
КОП				0	Адрес										

... с относительной адресацией

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
КОП				1	Режим			Смещение							

25 Безадресные команды

Безадресная команда

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	Расширение КОП											

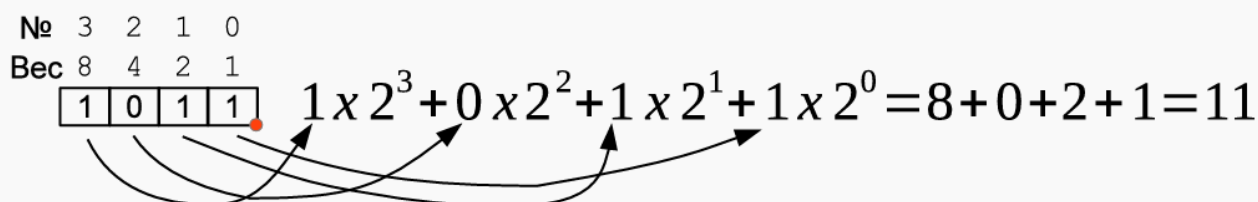
26 Представление чисел: фиксированная точка



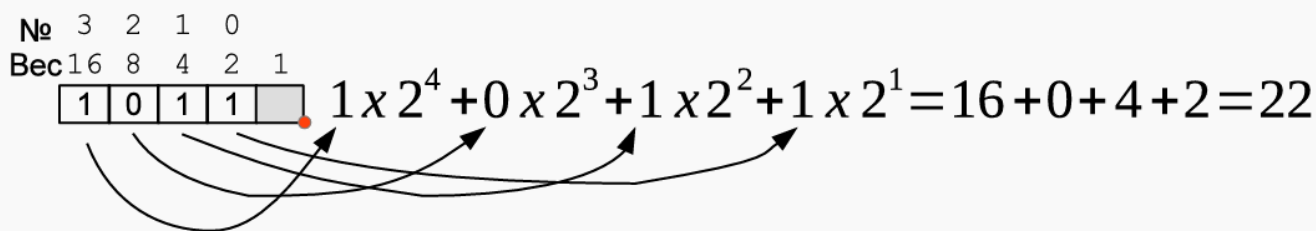
Представление чисел: фиксированная точка

- Целые: **двоичная** точка фиксирована за разрядом с номером 0, веса положительные

1) номер бита и степень веса разряда соответствуют



2) степень веса разряда ~ номеру бита + 1



73

В ЭВМ в основном используется три вида представления чисел — с фиксированной точкой, плавающей точкой и представление в виде строки символов. Особенностью представления чисел с фиксированной и плавающей точкой является то, что они должны размещаться в разрядной сетке ЭВМ (т. е. фактически ограничены определенным количеством двоичных разрядов). У каждого из разрядов определен номер разряда и вес разряда в двоичном представлении, который соответствует основанию системы счисления (2) в степени номера разряда. Само число формируется следующим образом: значение ячейки из каждого разряда умножается на вес, и для получения представления числа суммируются все полученные значения для каждого из разрядов. При помощи фиксирования двоичной точки возможно представление вещественных чисел. В этом случае мы можем фиксировать нашу двоичную точку после разряда с заданным номером, разделяя разрядную сетку на целую и дробную части числа.



Представление беззнаковых целых чисел

- Количество разрядов в разрядной сетке определяет область допустимых значений

- Минимальное 4-х разрядное беззнаковое число:

3	2	1	0
0	0	0	0

$$0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 0$$

- Максимальное 4-х разрядное беззнаковое число:

3	2	1	0
1	1	1	1

$$1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 15 = 2^4 - 1$$

- Диапазон:

$$0 \leq X \leq 2^4 - 1$$

- Диапазон для 16-ти разрядного беззнакового числа:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

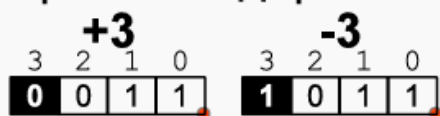
$$0 \leq X \leq 2^{16} - 1 = 65535_{75}$$



Представление знаковых целых чисел

- Нужно хранить признак знака числа достаточно 1-го бита, «0» значит «+», «1»=«-»

- Прямое кодирование (прямой код числа)



$$-7 = -(2^3 - 1) \leq X \leq 2^3 - 1 = 7$$

Двойной нуль!

- Дополнительный код

$$M = b^n - K$$

M — дополнение к числу K ($10 - 3 = 7$) !

b — основание системы счисления

n — количество разрядов



76

Прямой код в вычислительных устройствах напрямую не используется, т.к. необходима дополнительная логика при определении знаков и больших по модулю чисел при выполнении операций. Конструкция АЛУ при использовании прямого кода существенно усложняется.

29 Представление знаковых чисел: дополнительный код



Представление знаковых чисел: дополнительный код

$$M = b^n - K = ((b^n - 1) - K) + 1$$

Прямой код 5-ти разр. дес. чисел	Дополнительный код		
	5-ти разр. дес. чисел	4-х разр. шестн. чисел	16-ти разрядных двоичных чисел
-50000	50000		
-49999	50001		
-32768	67232	8000	1000 0000 0000 0000
-32767	67233	8001	1000 0000 0000 0001
-2	99998	FFFE	1111 1111 1111 1110
-1	99999	FFFF	1111 1111 1111 1111
0	00000	0000	0000 0000 0000 0000
1	00001	0001	0000 0000 0000 0001
32767	32767	7FFF	0111 1111 1111 1111
49999	49999		

K=+3

3 2 1 0

0 0 1 1

$M = b^n - K$

$2^4 - 3 = 13$

1 0 0 0 0

- 0 0 1 1

M = 1 1 0 1

ИНВ.

0 0 1 1

↓ ↓ ↓ ↓

1 1 0 0

+

1

M = 1 1 0 1

77

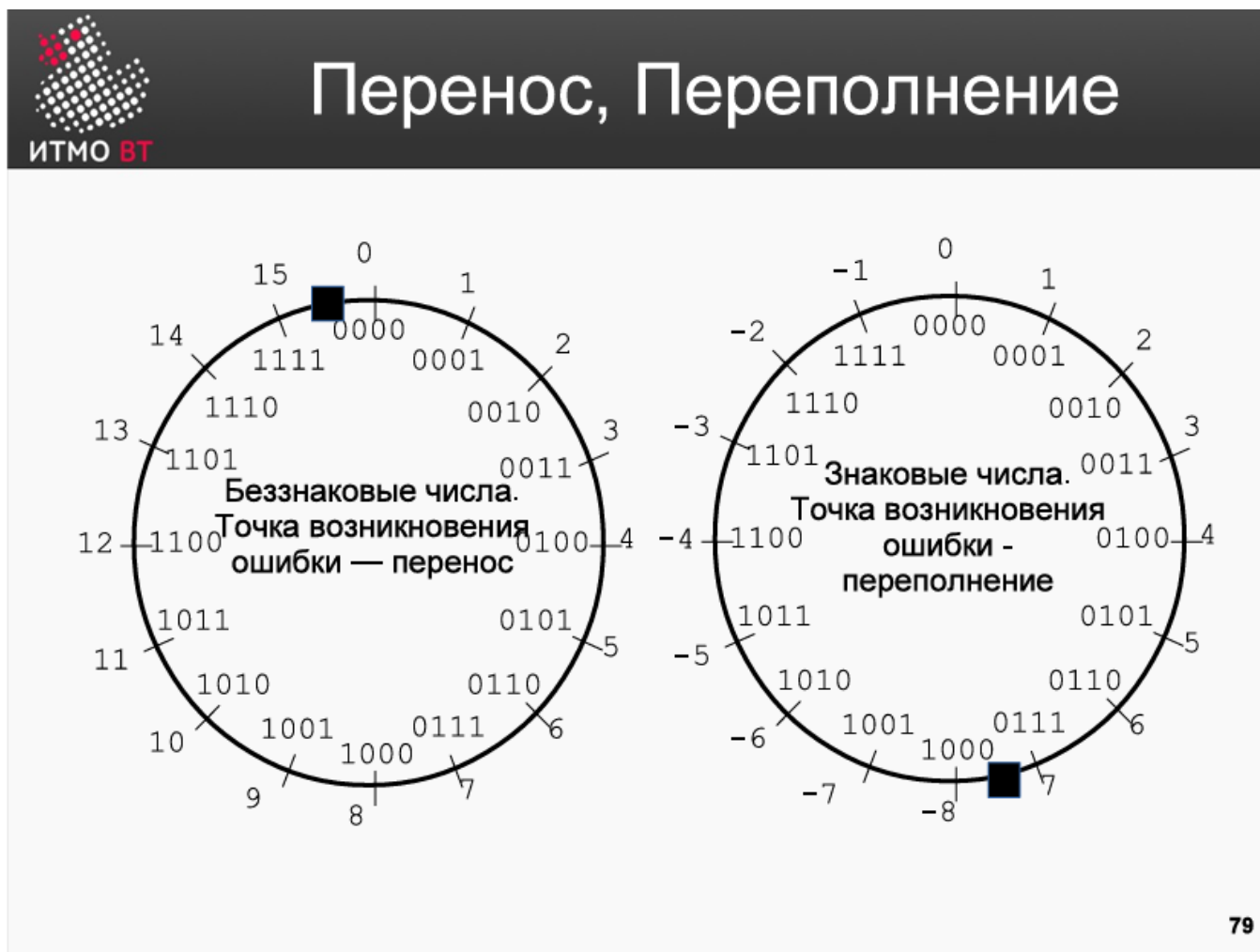
Для выполнения вычислений обычно используется Дополнительный код числа, который представляет отрицательные числа в виде дополнения до максимально возможного положительного числа + 1 в заданной разрядной сетке. Математически дополнительный код числа представлен в виде формулы, показанной на слайде. В двоичной системе получение поразрядного дополнения совпадает с простой инверсией битов числа. То есть, часть формулы $((b^n - 1) - K)$ просто заменяется инверсией разрядов числа K. В результате отрицательное число получается в 2 действия: первое — инверсия числа, второе — прибавление 1 к инверсии.

Получение дополнительного кода БЭВМ

Адрес	Содержимое		Комментарии
	Код	Мнемоника	
010	0200	CLA	X в аккумуляторе (2) Вычисление дополнения (инверсия битов - FFFD) Инкремент (FFFE) Сохранение результата
011	4016	ADD 16	
012	0280	NOT	
013	0700	INC	
014	E017	ST 17	
015	0100	HLT	X -X
016	0002	X	
017	FFFE	R	

Да, я знаю, все это можно было сделать проще!

-32768	8000	1000 0000 0000 0000
-32767	8001	1000 0000 0000 0001
-2	FFFE	1111 1111 1111 1110
-1	FFFF	1111 1111 1111 1111
0	0000	0000 0000 0000 0000
1	0001	0000 0000 0000 0001
32767	7FFF	0111 1111 1111 1111



В беззнаковых числах, если прибавить 1 к 15 или вычесть из 0 единицу произойдет потеря значения числа, которая сопровождается возникновением переноса. Этот перенос в конструкции ЭВМ учитывается в бите C (Carry), который служит сигналом программисту, что произошла ситуация потеря значения, и ее необходимо обработать отдельно. В знаковых числах точка возникновения ошибки расположена между представлением чисел -8 и 7. При вычитании из -8 единицы или прибавления единицы к 7 возникает переполнение (Overflow), которое в ЭВМ контролируется битом переполнения. Напомним, что данный бит обозначается в БЭВМ - V, а в разных современных архитектурах процессоров O, V или OV. АЛУ определяет переполнение по следующему правилу: если поразрядные переносы в знаковом и старшем разряде одновременно отсутствуют или присутствуют - значит переполнения нет, если присутствует только в одном — значит переполнение знаковой разрядной сетки есть.

1101 ← биты совпадают
+1101 -3
0101 +5

10010 2
C=1 V=0

1000 ← биты различны
+1000 -8
1111 -1

10111 7 (≠-9) ←ошибка
C=1 **V=1**

0111 ← биты различны
+0111 7
0001 1

01000 -8 (≠8) ←ошибка
C=0 V=1



Представление беззнаковых целых чисел

- Количество разрядов в разрядной сетке определяет область допустимых значений

- Минимальное 4-х разрядное беззнаковое число:

3	2	1	0
0	0	0	0

$$0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 0$$

- Максимальное 4-х разрядное беззнаковое число:

3	2	1	0
1	1	1	1

$$1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 15 = 2^4 - 1$$

- Диапазон:

$$0 \leq X \leq 2^4 - 1$$

- Диапазон для 16-ти разрядного беззнакового числа:

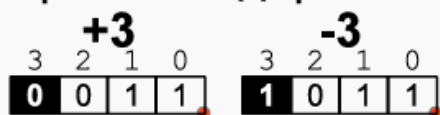
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

$$0 \leq X \leq 2^{16} - 1 = 65535_{75}$$

Внутри БЭВМ, внутри той же самой разрядной сетки можно представить знаковые и беззнаковые числа. На слайде показано представление беззнаковых чисел. На примере четырехразрядной сетки видно, что минимальным беззнаковым числом может быть 0, а максимальным - 15. В БЭВМ максимальное значение для 16-ти разрядного беззнакового числа определено на слайде. Минимальное значение числа составляет 0, а максимальное - 65535.

- Нужно хранить признак знака числа достаточно 1-го бита, «0» значит «+», «1»=«-»

- Прямое кодирование (прямой код числа)



$$-7 = -(2^3 - 1) \leq X \leq 2^3 - 1 = 7$$

Двойной нуль!

- Дополнительный код

$$M = b^n - K$$

M — дополнение к числу K ($10-3=7$) !

b — основание системы счисления

n — количество разрядов



76

Представление чисел со знаком более сложно. Существует несколько способов кодирования знаковых чисел.

Прямое кодирование. Знак можно хранить, записав его в отдельный, обычно самый старший, разряд слова. Общепринято, что знак кодируется 0 если число положительное и 1 если отрицательное. Прямой код в вычислительных устройствах напрямую не используется, т.к. необходима дополнительная логика при определении знаков и больших по модулю чисел при выполнении операций. Конструкция АЛУ при использовании прямого кода существенно усложняется.

Для выполнения вычислений обычно используется дополнительный код числа, который представляет отрицательные числа в виде инверсии всех битов +1 (в двоичной сс). Также математически дополнительный код числа представлен в виде формулы, показанной на слайде. Дополнительный код можно использовать для любой позиционной системы счисления.

Получение дополнительного кода БЭВМ

Адрес	Содержимое		Комментарии
	Код	Мнемоника	
010	0200	CLA	X в аккумуляторе (2) Вычисление дополнения (инверсия битов - FFFD) Инкремент (FFFE) Сохранение результата
011	4016	ADD 16	
012	0280	NOT	
013	0700	INC	
014	E017	ST 17	
015	0100	HLT	X -X
016	0002	X	
017	FFFE	R	

Да, я знаю, все это можно было сделать проще!

-32768	8000	1000 0000 0000 0000
-32767	8001	1000 0000 0000 0001
-2	FFFE	1111 1111 1111 1110
-1	FFFF	1111 1111 1111 1111
0	0000	0000 0000 0000 0000
1	0001	0000 0000 0000 0001
32767	7FFF	0111 1111 1111 1111

Представление в разрядной сетке	Беззнаковые числа	Знаковые числа
0000 0000 0000 0000	0	0
0000 0000 0000 0001	1	1
...		
0111 1111 1111 1110	32766	32766
0111 1111 1111 1111	32767	32767
1000 0000 0000 0000	32768	-32768
1000 0000 0000 0001	32769	-32767
1111 1111 1111 1110	65534	-2
1111 1111 1111 1111	65535	-1

ОДЗ:

$$\downarrow$$

$$0 \leq X \leq 2^{16} - 1$$

$$\downarrow$$

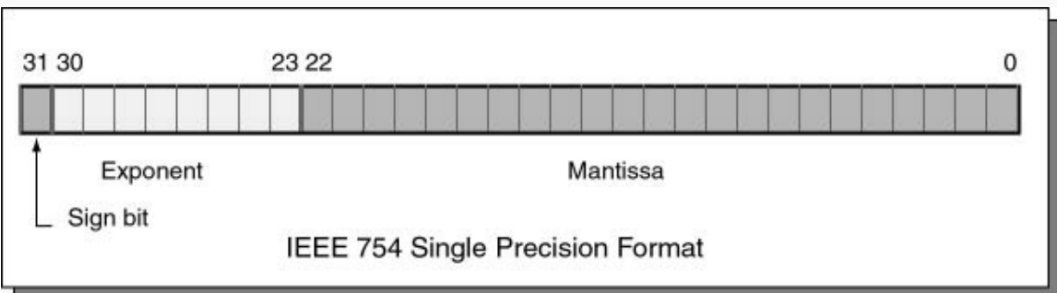
$$-2^{15} \leq X \leq 2^{15} - 1$$

32 Представление чисел с плавающей точкой



Представление чисел с плавающей точкой

В БЭВМ — НЕТ!


$$X = (-1)^{(sign)} \times (1 + Mantissa) \times 2^{(exponent - 127)}$$

81

Число с плавающей точкой — это численное представление вещественного числа в программировании, которое является его приближённым значением.

Двоичное представление чисел с плавающей точкой содержит три части:

1. Знаковый бит. Указывает, положительное число или отрицательное.
2. Экспонента. Показывает, на какое число нужно умножать мантиссу.
3. Мантисса. Фиксированное количество битов, которое выражает точность числа.

Числа с плавающей точкой представлены в стандарте IEEE 754. Этот стандарт нужен, чтобы разные компьютерные архитектуры могли одинаково удобно и эффективно работать с вещественными числами.

БЭВМ не умеет на уровне машинных команд работать с числами с плавающей точкой. Представление логической и символьной информации не отличаются от современных ЭВМ и выходят за рамки данных методических указаний.

33 Представление логической информации



Представление логической информации

- 1-true, 0-false
- 16-ти разрядное число содержит 16 логических значений

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

ОДЗ: $X_i \in \{0,1\}$ где $0 \leq i \leq 15$

83

Логическая информация представлена в БЭВМ как показано на слайде.

В 16-ти разрядной сетке БЭВМ каждому биту из логической информации соответствует значению: либо true, либо false. При этом общепринято, что 1 это истина (true), а 0 = ложь или false. И таких операндов, соответственно, 16.

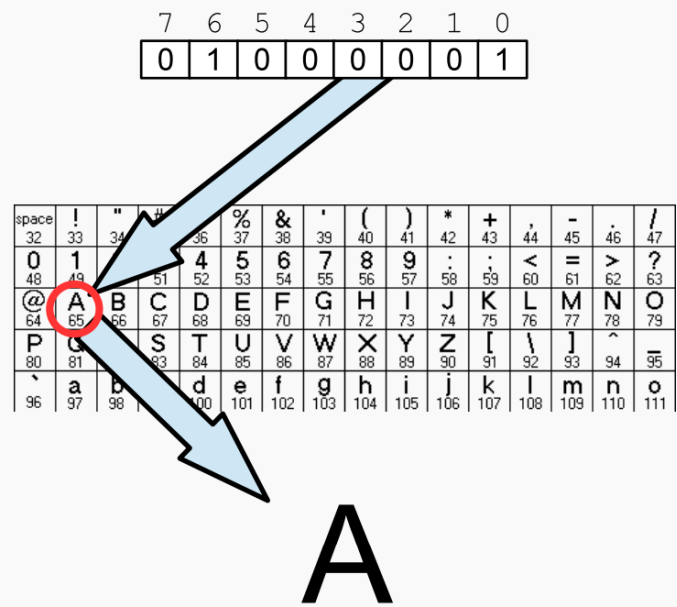
Когда применяется команда AND (поразрядное И) в БЭВМ, то происходит логическое умножение между одной ячейкой памяти и аккумулятором. Соответственно, будет произведено 16 логических операций одновременно. Полученный результат является результатом логической операции, и в большинстве случаев его нельзя трактовать, как математический (т. е. знаковый или беззнаковый, это просто 16 отдельных логических значений).

Область допустимых значений для логических переменных представлена на слайде.

34 Представление символьной и текстовой информации



Представление символьной и текстовой информации



The diagram illustrates the binary representation of the letter 'A'. At the top, a byte is shown with bits 7 through 0: 0 1 0 0 0 0 0 1. A large blue arrow points from this byte to the ASCII table below. In the ASCII table, the letter 'A' is circled in red, and its decimal code, 65, is highlighted. Another blue arrow points from the circled 'A' to a large 'A' character at the bottom.

space	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
0	1	2	3	4	5	6	7	8	9	:	:	<	=	>	?
48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111

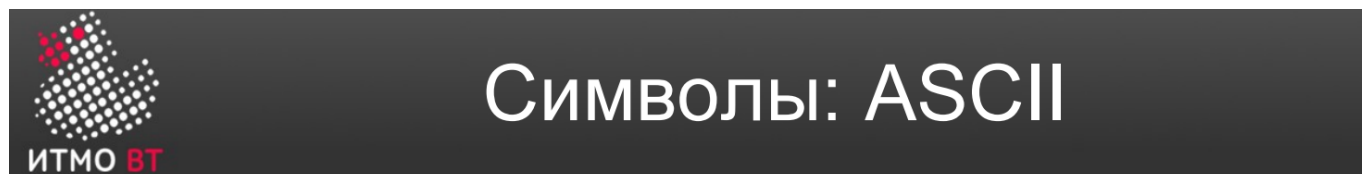
Представление текстовой информации в вычислительных машинах основано на кодировании букв алфавитов для существующих на земле языков, которые традиционно используются человечеством. Символ - это графическое изображение, которое используется человеком для создания слов, текстов и другой значимой информации. Как известно, к символам относятся буквы, знаки препинания, символы валют, цифры и т.д.

В ЭВМ представление текстовой информации основывается на кодировании букв и символов при помощи кодовой таблицы. Кодовая таблица (или кодировка символов) является соглашением между разработчиками о соответствии каждому символу определенного порядкового номера или кода, чтобы его можно было сохранять в памяти ЭВМ или передавать по каналам связи. Пример такой кодовой таблицы представлен на слайде. Например, если нам необходимо закодировать латинскую букву «А», то по таблице мы можем определить, что этой букве соответствует код 65. Если нам необходимо закодировать буквами слово, то как и в написании этого слова, коды букв в памяти будут располагаться последовательно, например слово DEDA будет представлено последовательностью кодов 69, 68, 69, 65, которые можно представить в ЭВМ, например, в виде целых беззнаковых чисел размером в 1 байт. Для работы пользователя с символьной информацией недостаточно сохранять в памяти коды символов. Необходимо еще и отображать текстовую информацию на экране (или других устройствах вывода) компьютера. Для хранения графических начертаний символов в ЭВМ существуют шрифты (fonts). Они хранят все графические изображения символов в заданном алфавите, и обычно устанавливаются вместе с операционной системой, драйвером принтера или микропрограммой BIOS. В шрифтах каждому коду символа соответствует свое начертание. Отдельно хранятся изображения строчных и заглавных букв, цифр, специальных символов и т. д.

Шрифты бывают векторные и растровые. В растровых шрифтах каждому коду символа соответ-

ствуется изображение, определенного (в точках) размера. В векторных хранится принцип начертания символа в виде последовательности линий.

35 Символы: ASCII



ASCII Code Chart

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

7 БИТ! Старший — для контроля четности

88

Как отмечено на предыдущем слайде, символы кодируются с использованием кодовой таблицы. Первая кодировка содержала 32 символа, и была предназначена для использования в телеграфе. Легко заметить, что символы этой кодировки можно было упаковать в пять бит. Следующей важной кодировкой символов, коды символов которой можно увидеть во всех современных кодировках стала кодовая таблица ASCII. ASCII расшифровывается как American Standard Code for Information Interchange. Таблица представлена на слайде. Как можно видеть, в ней присутствуют заглавные и прописные буквы, цифры, знаки препинания, денежные знаки, спецсимволы. В первых 32 позициях кодировки расположены управляющие символы, которые используются в качестве служебных, например, для перемещения курсора на терминале и организации текстов. Организация текста внутри памяти вычислительной системы обычно совпадает с организацией файла формата «.txt», который, в свою очередь, был придуман исходя от принципа работы пишущей машинки (первые консоли ЭВМ и были пишущими машинками). Поэтому, в кодовой таблице есть символы перевода строки (LF), возврата на один символ (BS), возврата каретки (CR) и другие. Символ NUL в современных системах часто используется для признака окончания строки. С помощью символа BS на принтере можно печатать один символ поверх другого. В ASCII таким способом можно добавить к буквам диакритические знаки. Пример: а BS ' → á. Отметим, что разные ОС по-разному кодируют конец строки. Это связано с особенностью работы печатных машинок и телетайпов. UNIX системы используют один символ — CR (при этом виртуальная печатная машинка как бы переводит печатную консоль в начало

и автоматически на следующую строку), а Windows - последовательностью из двух: перенос строк и возврат каретки. Из-за этого текстовые файлы между этими ОС не полностью совместимы. В ASCII символы кодировались семью разрядами, 8-й (старший) разряд не использовался для кодировки символов, а использовался для контроля четности битов передаваемого по каналам связи символа. При этом на передающей стороне считалось количество единичных битов в коде символа и если оно было четным, то старший разряд устанавливался в единицу. На приемной стороне также подсчитывалась четность количества единичных битов кода символа и проверялось на совпадение с старшим битом символа. Если эти разряды не совпадали, это служило признаком ошибки передачи.

36 Символы: ASCII (КОИ-7Н0)КОИ-7Н1 (РУС),КОИ7Н2 (Mix)

КОИ - Код Обмена Информацией **КОИ-7Н0** - Совпадал с ASCII (но символ доллара был заменён на , всё таки не место вонючий капиталист доллар в великий коммунист страна (это СКРИНШОТ, символ так и выглядит!, кружочек с палками, проведёнными от центра)) **КОИ-7Н1** - все латинские заменены на русские **КОИ-7Н2** - Заглавные русские и английские буквы, причём буквы кириллицы располагались на месте максимально похожих на себя английских букв: А - А В - Б С - Ц D - Д Е - Е F - Ф G - Г Н - Х I - И J - Й K - К L - Л M - М N - Н и т.д.

Отсюда возникло понятие транслита. Случилось так, потому что многие программы брались у США и необходимо было выводить программы текстом так, чтобы в случае отсутствия в системе русских шрифтов текст, пусть и с неудобствами в восприятии, всегда можно было прочитать с помощью английских шрифтов.

Очевидный минус - порядок букв, сортировка крайне неудобна. Для сортировки нужна была отдельная таблица, содержащая алфавитный порядок букв.

37 Символы: КОИ-8

Добавился 8 бит с КОИ-7Н*

Когда появилась Extended ASCII было принято решение для русских букв строчных и заглавных букв использовать позиции верхней части таблицы. Также в верхней части были различные символы для рисования таблиц.

Содержала и русские и английские буквы, но буквы сохранился с КОИ-7Н1, КОИ-7Н2. Т.е., если попадалась старая программа, её всё ещё можно было прочитать (русские символы перемещались в младшую часть таблицы).

До сих пор является стандартом для обмена электронной почтой.

Очевидный минус - порядок букв, сортировка крайне неудобна.

38 Символы: ISO8859-5 (ГОСТ-основная)

Была создана в 1988 году на базе “основной кодировки”. Для русского языка было предложено две стандартные кодировки ISO8859-5 (ГОСТ-основная) и cp866 (ГОСТ-альтернативная, кодировка DOS).

Широко применяется в Сербии и иногда в Болгарии на unix-like системах, в России в настоящее время почти не употребляется, правда, на некоторых коммерческих системах для русского языка до сих пор используется ISO8859-5: Solaris, AIX, HP-UX.

До широкого использования UNICODE в российском сегменте велись “войны кодировок”. По факту используется 5 таблиц. Совместно с КОИ-8 использовались ISO8859-5 и cp866 (ГОСТ-альтернативная), WIN-1251, кириллическая кодировка для машин Macintosh.

Преимущества по сравнению с КОИ-7: русские буквы по алфавиту символ занимает 1 байт

39 Символы: WIN1251

На смену кодировкам ГОСТ-* пришла кодировка WIN1251 от Microsoft.

Микромягкие как всегда. Решили использовать не CP86 из DOS, а придумать новую, на базе кодировок, использовавшихся в ранних “самопальных” русификаторах Windows, совместно с представителями “Параграфа”, “Диалога” и российского отделения Microsoft в 1990-91 годах несмотря на стандартизацию кириллических символов.

40 Символы: UNICODE, UTF-8

В настоящее время повсеместно используется таблица кодировки UNICODE. Стандарт был предложен в 1991 году некоммерческой организацией “Консорциум Юникода” (Unicode Consortium). Позволяет закодировать очень большое число символов из разных систем письменности. Могут соседствовать китайские, японские иероглифы, математические символы, буквы греческого алфавита, символы музыкальной нотации и т.д., и всё это без переключения кодовых страниц.

Состоит из двух частей: универсального набора символов (Universal Character Set, UCS) и нескольких форм представления кодировок (Unicode Transformation Format, UTF). Универсальный набор символов перечисляет допустимые по стандарту юникод символы и присваивает каждому код в неотрицательно целого числа, записываемого в 16-ричной форме с префиксом U+ (U+040F). Семейство кодировок определяет способы преобразования кодов символов для передачи в потоке или в файле.

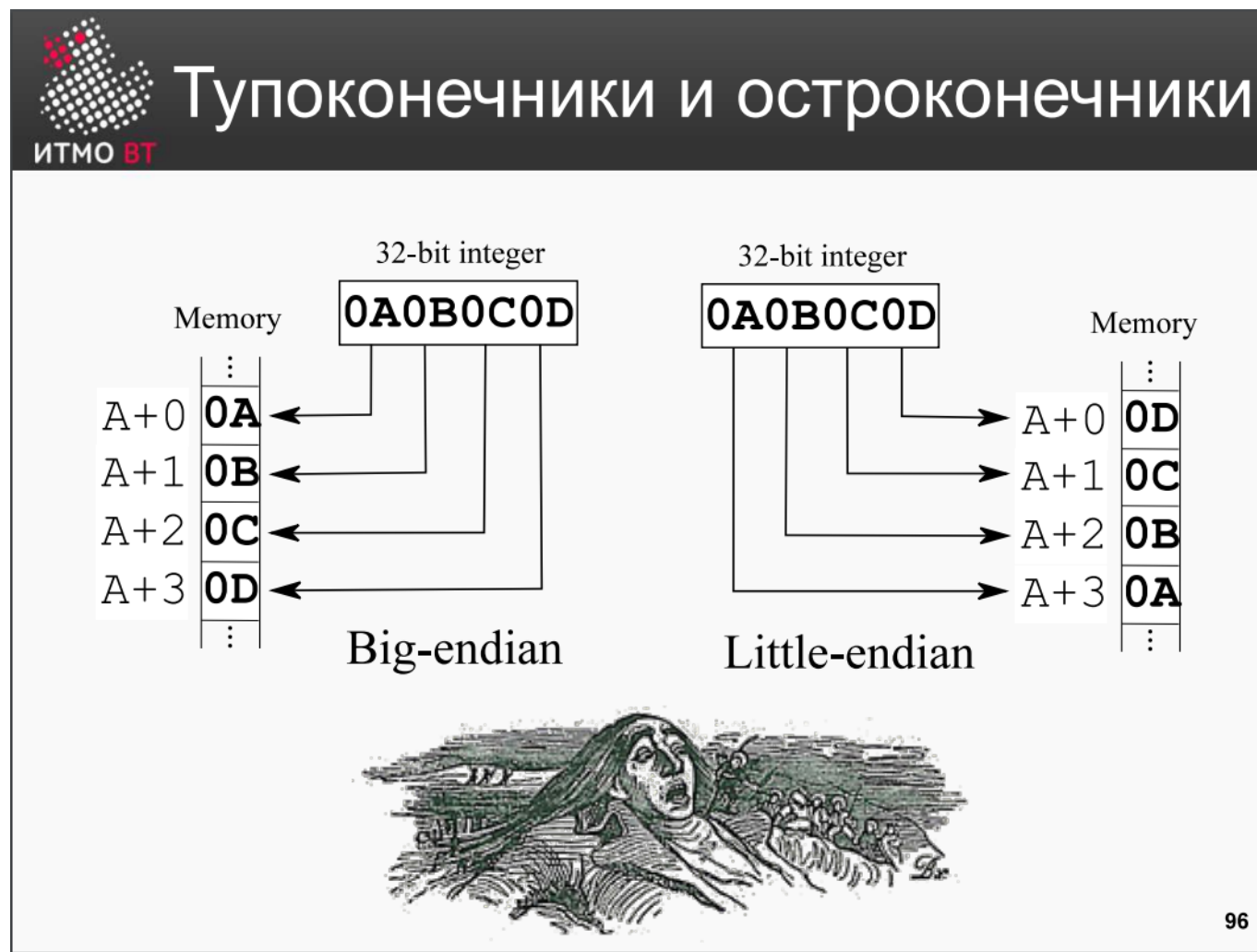
В начальной версии полностью совпадал с ASCII. Младшие 7 битов предвзяются битом 0 и далее следует 7 битов символа. Если символ превышает 7 разрядов, то он представляется в виде двух байтов. При этом первые 5 бит предвзяются преамбулой 110, следующий байт идёт с преамбулой 10, остальные 6 копируются во второй байт. Если превышает 11 бит, то по следующему принципу:

Буква «А» → unicode \u0410 → 0000 0100 0001 0000
→ UTF-8 110 1 0000 1001 0000 → D0 90

Основная проблема UTF-8 состоит в том, что один символ может быть закодирован 1-4 байтами. Наиболее часто используемая операция со строками - это определение их длины. В UTF-8 ее сложнее определить, чем для любой однобайтной кодировки, где длина строки равна количеству байтов. Подсчёт количества символов для UNICODE реализован системными библиотеками.

Недостаток несоответствия длины решается путём использования UTF-16 или UTF-32, но в таком случае символы будут больше весить.

41 Big-endian и Little-endian



Big Endian и Little Endian — это два разных способа упорядочивания байтов в памяти для представления данных, таких как числа. Эти термины описывают, какой байт (старший или младший) располагается первым (то есть, по самому младшему адресу).

Big Endian - Старший байт хранится в памяти по самому младшему адресу. **Little Endian** - Младший байт хранится в памяти по самому младшему адресу.

42 Представление строк

NUL-terminated строки - Это строки, завершающиеся специальным символом NUL

Строки с упаковкой длиной - В этом подходе длина строки хранится отдельно от её содержимого (или в самом начале)

UTF-8 может доставить еще больше радости программисту для определения количества байт, хранимых в строке, и определения количества символов в ней.

1) NUL terminated String

«Little-endian»

53 ₁₆	74 ₁₆	72 ₁₆	69 ₁₆	6E ₁₆	67 ₁₆	00	??
слово 1		слово 2		слово 3		слово 4	

«Big-endian»

74 ₁₆	53 ₁₆	69 ₁₆	72 ₁₆	67 ₁₆	6E ₁₆	00	??
слово 1		слово 2		слово 3		слово 4	



2) Упаковка с длиной (как в Паскале)

0006	74 ₁₆	53 ₁₆	69 ₁₆	72 ₁₆	67 ₁₆	6E ₁₆
слово 1	слово 2		слово 3		слово 4	



История

- Нулевое поколение — механические компьютеры (1642–1945)
 - Налоговый сумматор (Паскаль), калькулятор на 4 действия (Лейбниц)
- Первое поколение — электронные лампы (1945–1955)
 - COLOSSUS (1943, Тьюринг), ENIAC (1946, Моушли), IAS (1951, фон Нейман)
- Второе поколение — транзисторы (1955–1965)
 - TX-0 (1955, МТИ), PDP-1 (1961, DEC), PDP-8, 7090 (IBM), 6600 (1964, CDC)
- Третье поколение — интегральные схемы (1965–1980)
 - Семейство System/360 (1965, IBM), PDP-11 (1970, DEC)
- Четвертое поколение — сверхбольшие интегральные схемы (1980–?)
 - IBM PC (1981), Apple, Intel, IBM, Dec, Compaq, HP, Sun...
- Пятое поколение — небольшие и «невидимые» компьютеры (1989–?)



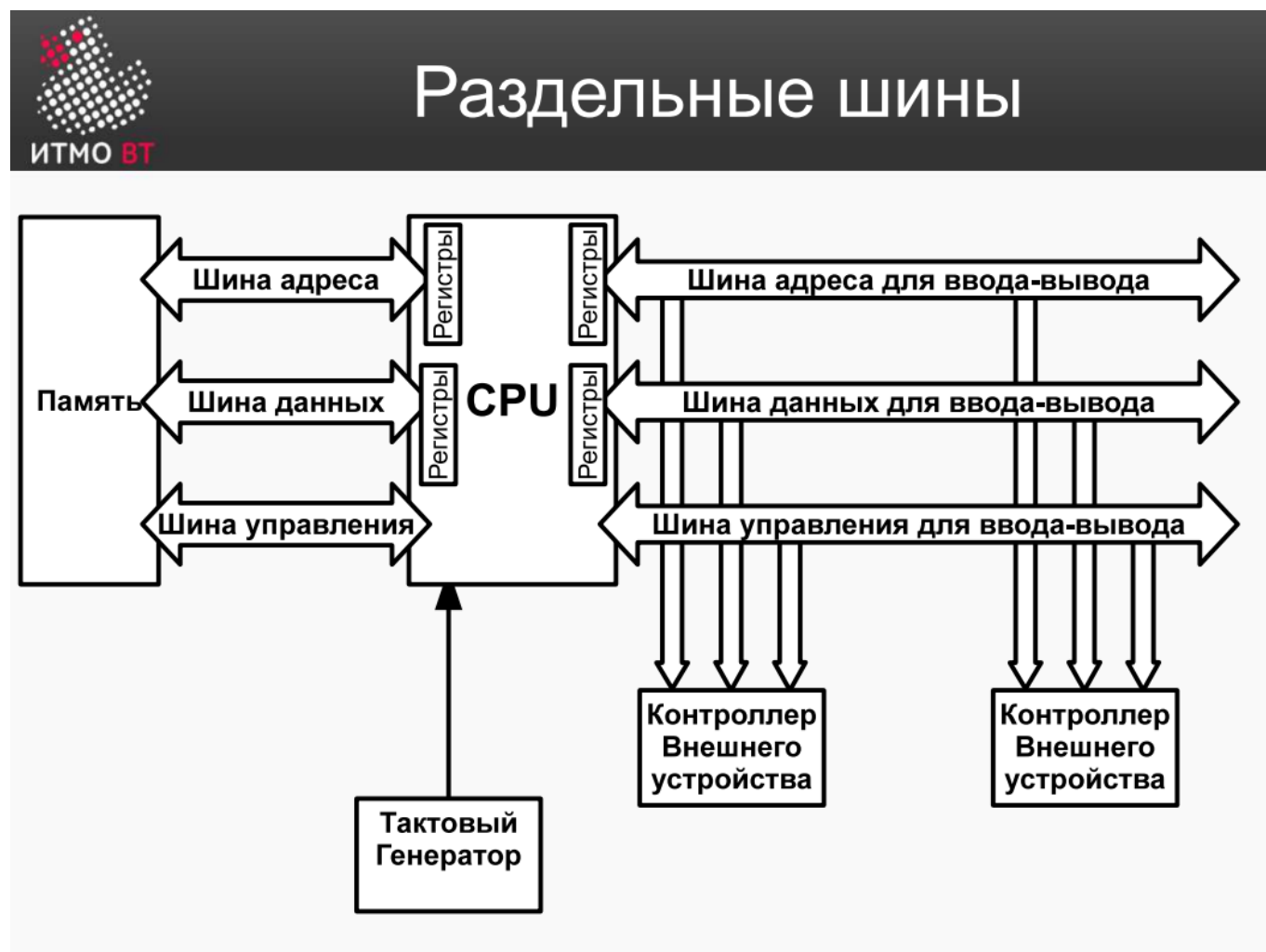
История в СССР/России

- Первое поколение — электронные лампы
 - Лебедев, 1950, МЭСМ
 - БЭСМ, 1953, БЭСМ — 10000 оп/с, 53КВТ.
- Второе поколение — транзисторы
 - 5Э926, 1964, самодиагностика, горячая замена, 500000 оп/с
 - БЭСМ-6, 1965 год, +ковейерная обработка, удаленное управление по телеф. Линиями
- Третье поколение — интегральные схемы
 - Директива «Ряд», 1968 год, клонирование S/360, 1971 год — ЕС ЭВМ
 - Клоны PDP-11
- Четвертое поколение — сверхбольшие интегральные схемы
 - Эльбрус — разработка по настоящее время

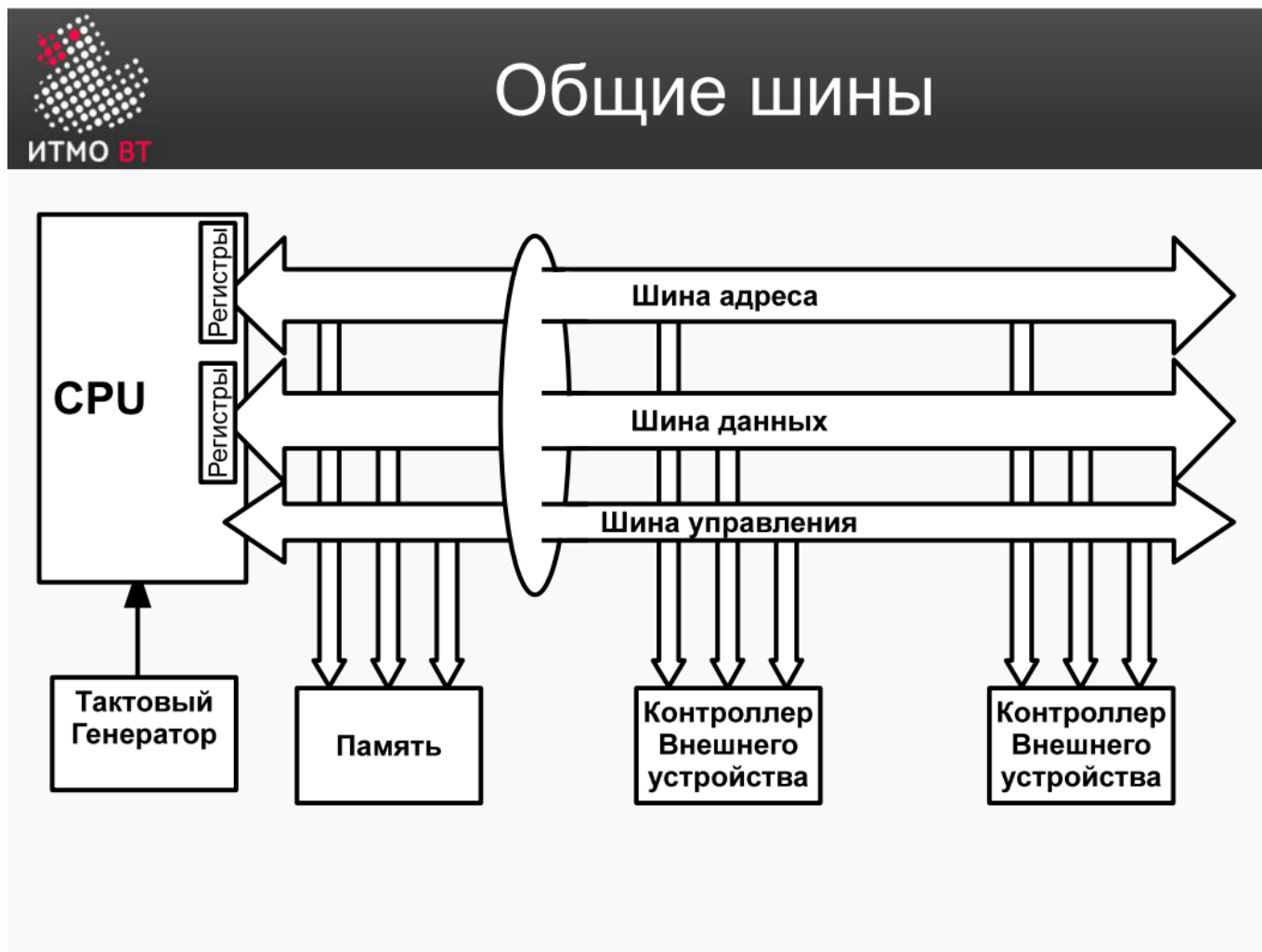
<http://statehistory.ru/3932/Istoriya-razvitiya-sovetskikh-EVM-do-1980-go-goda/>



Канальная организация — это способ построения вычислительных систем или их подсистем, при котором операции ввода-вывода или передачи данных между устройствами выполняются с использованием специализированных каналов. Эти каналы предоставляют выделенные пути передачи данных, что повышает производительность и снижает нагрузку на центральный процессор (ЦП).



Производительность процессора стала больше чем памяти, поэтому процессор занял центральную позицию. Он управлял памятью под тремя различными шинами. Скорость процессора и памяти повысилась, а контроллер внешних устройств остался примерно таким же

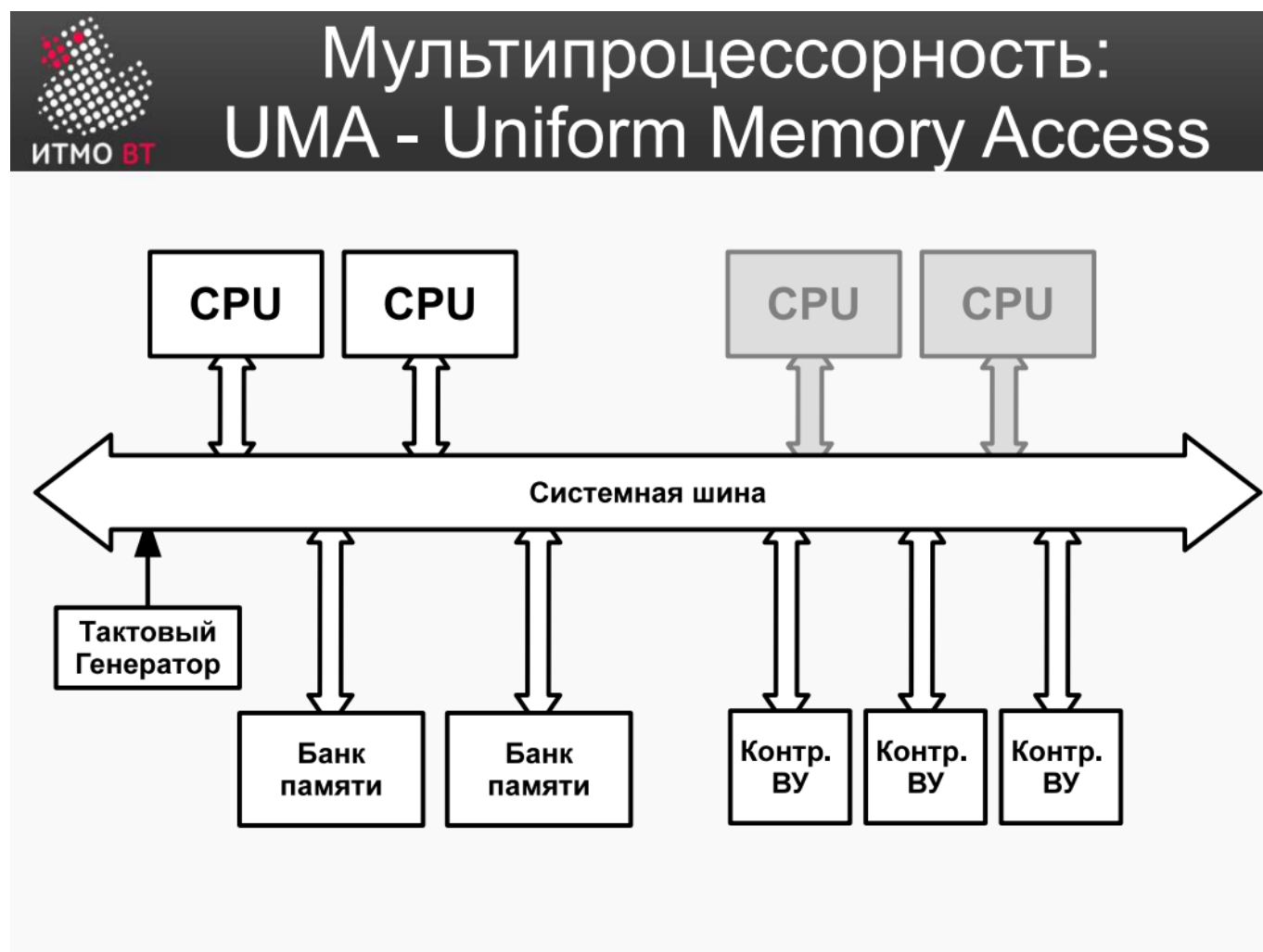


Отличием между 1,2 и третьим поколением является задача увеличения количества элементов и производительности на единицу объема. Для упрощения конструкции на одной шине стали располагаться все устройства кроме процессора (т.к. с памятью работаем быстрее, а к контроллерам обращаемся реже).

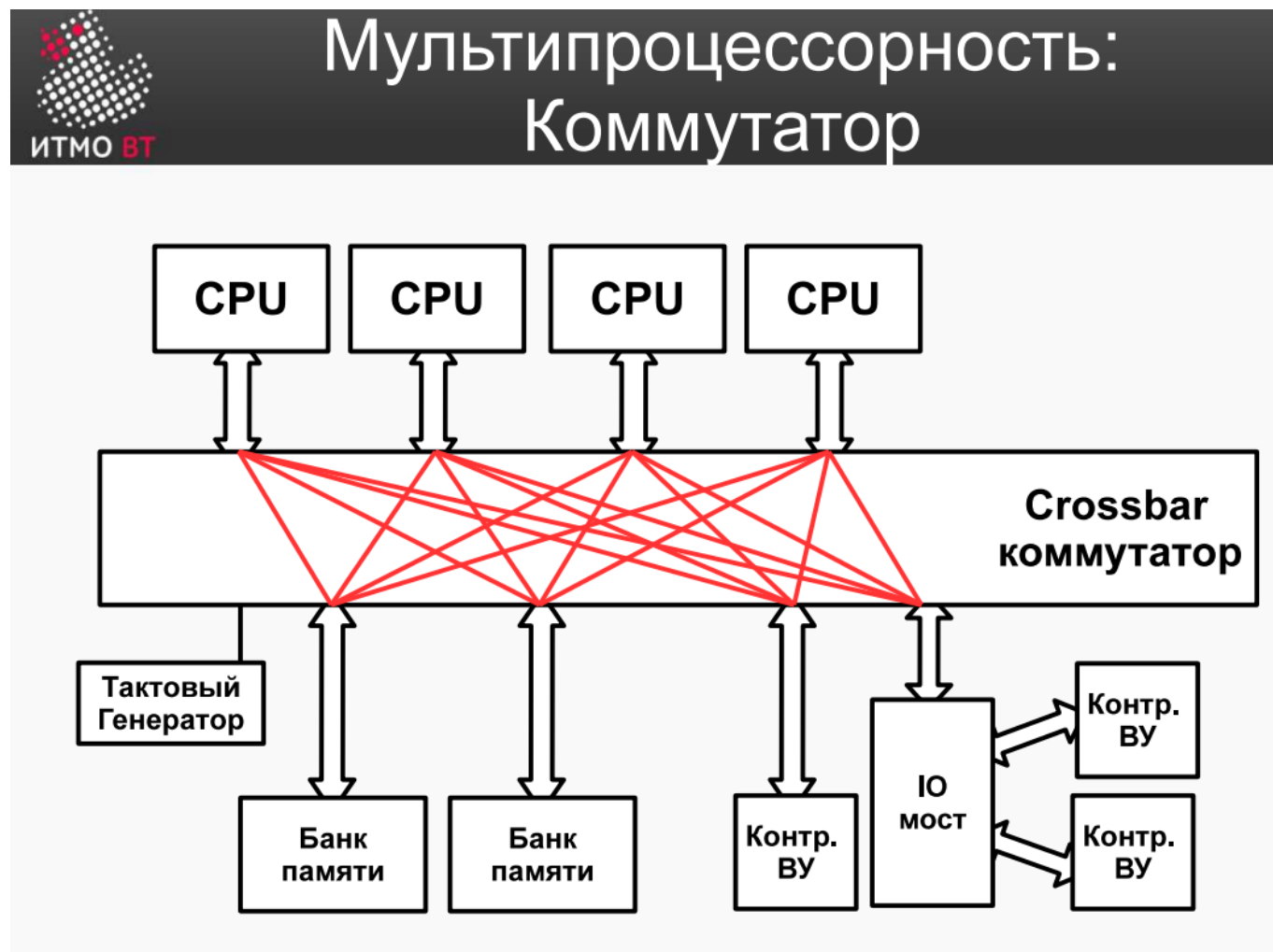


Для сокращения количества проводов, которые соединяли блоки вычислительной машины, адрес и данные стали передавать по одним и тем же физическим проводам. В один импульс тактового генератора передавался адрес, в другой – данные

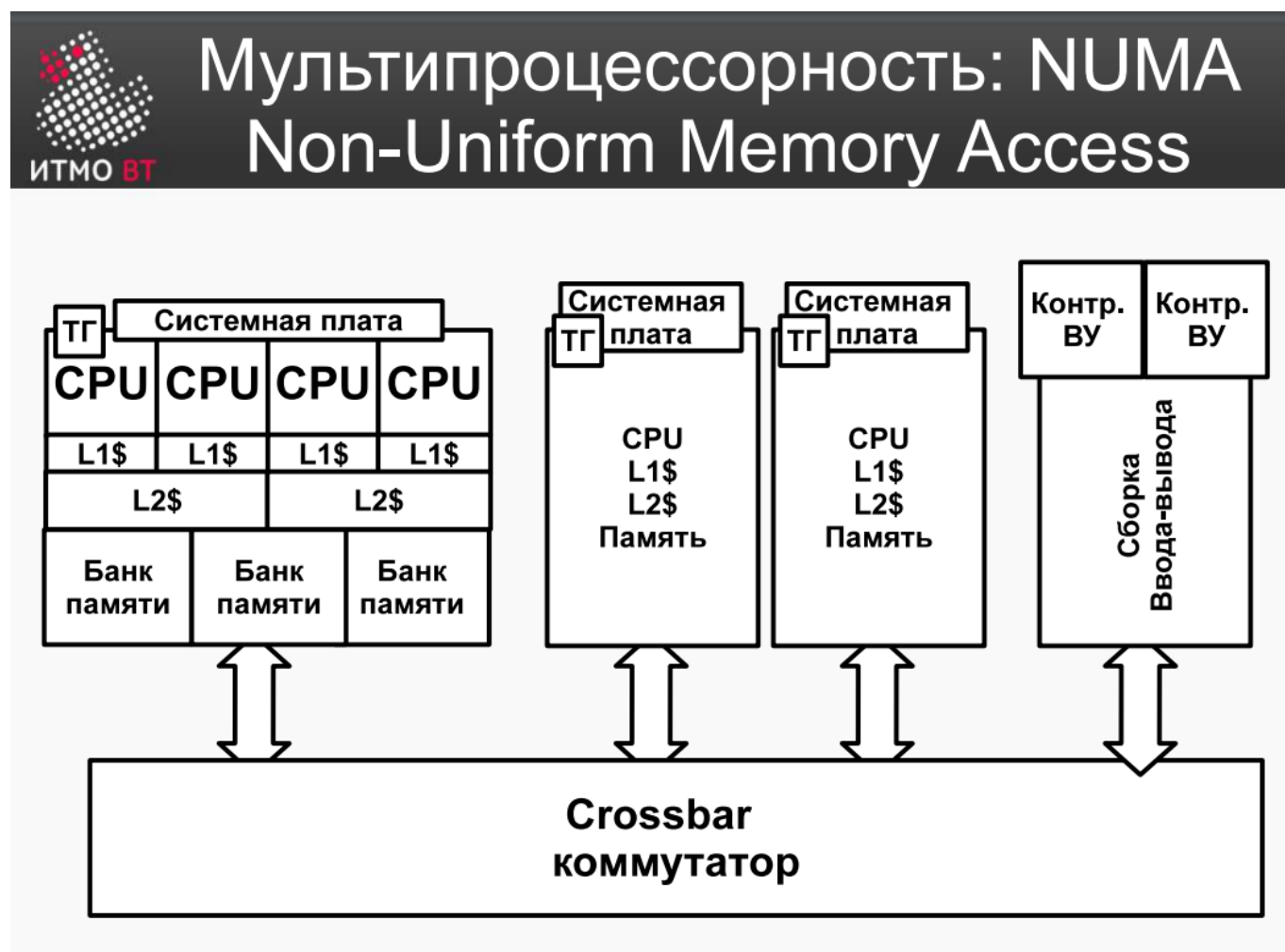
49 Мультипроцессорность:UMA - Uniform Memory Access



UMA – архитектура современных машин. Технически UMA-системы предполагают наличие узла, соединяющего каждый из n процессоров с каждым из m модулей памяти. Простейший путь построения таких ВС - объединение нескольких процессоров с единой памятью посредством общей шины. В каждый момент времени обмен по шине может вести только один из процессоров. При использовании системной шины с несколькими процессорами возникла нехватка пропускной способности. Само использование нескольких процессоров обеспечивает возможность параллельной организации многих потоков команд и данных.



Коммутатор представляет собой несколько системных шин, каждая из которых подключена к отдельному процессору и остальным устройствам. Он в состоянии параллельно обслуживать несколько запросов. Каждый процессор может быть соединен со своим модулем памяти и иметь доступ к нему на максимальной скорости. Соперничество между процессорами может возникнуть при попытке одновременного доступа к одному и тому же банку памяти. В этом случае доступ получает только один процессор, а прочие - блокируются. Произошло деление устройств ВУ на быстродействующие и медленнодействующие



NUMA (Non-Uniform Memory Access) — схема реализации компьютерной памяти, используемая в мультипроцессорных системах, когда время доступа к памяти определяется её расположением по отношению к процессору. Принцип работы: процессор имеет быстрый доступ к локальной памяти через свой контроллер, а также более медленный канал до памяти, подключённой к контроллерам (слотам) других процессоров, реализуемый через компьютерную шину обмена данными. Преимущества NUMA: быстрое перемещение данных и низкая задержка в мультипроцессорной системе. Кроме того, NUMA уменьшает репликацию данных и упрощает программирование.



Современные коммерческие процессоры

- Разрядность адреса и данных 16/32/64 бита
- Тактовые частоты 500МГц-5ГГц.
- Многопроцессорные 1-100+ CPU
- Многоядерные 1-16 ядер
- От 1 ГБ до терабайтов ОЗУ
- Используют кэш-память разных уровней
- Суперскалярные
- CISC, RISC, VLIW

Объяснение отличий современных процессоров:

- 64-битная архитектура. Была получена обратная совместимость с 16- и 32-разрядными приложениями.
- В современных коммерческих процессорах используется многоуровневая система кэш-памяти:
 - Кэш-память первого уровня (L1). Это самая маленькая, но и самая быстрая область кэш-памяти. Её объём не превышает пару десятков килобайт. В L1 содержатся данные, которые чаще всего используются процессором
 - Кэш-память второго уровня (L2). Чуть медленнее кэш-памяти L1, но и объём её в современных процессорах измеряется уже в мегабайтах. Служит она для временного хранения важных данных, вероятность запроса которых ниже, чем у данных, находящихся в L1 кэше.
 - Кэш-память третьего уровня (L3). Ещё более объёмная и ещё более медленная. Но она всё равно быстрее любой оперативной памяти. Её размер в современных процессорах достигает нескольких десятков мегабайт. В отличие от L1 и L2, кэш третьего уровня является общим для всех ядер процессора. L3 кэш служит для временного хранения важной информации с относительно низкой вероятностью запроса, а также для обеспечения обменом данными между ядрами процессора
- Суперскалярность современных коммерческих процессоров заключается в их способности выполнять более одной простой (скалярной) операции за один такт, что достигается благодаря включению в их состав нескольких параллельно работающих исполнительных устройств.



CISC, RISC, VLIW

- **Complex Instruction Set Computer**
 - Традиционные процессоры (например Intel), отягощенные совместимостью
 - **Reduced Instruction Set Computer**
 - Простой набор инструкций, выполнение инструкции за такт
 - **Very Long Instructions Word**
 - Несколько инструкций, упакованных в одну команду
 - Упаковка операций в инструкцию ложится на компилятор
-
- CISC (Complex Instruction Set Computer) — тип процессорной архитектуры, в которой используются сложные и разнообразные наборы команд.
 - Главная идея состоит в том, чтобы минимизировать количество инструкций на программу, даже если для этого нужно усложнять сами команды. Команды CISC часто выполняют несколько операций за один раз. Процессоры с такой структурой имеют сложные декодеры и выполняют команды за большее число тактов.
 - Преимущества CISC-архитектуры: компактность наборов инструкций уменьшает размер программ и количество обращений к памяти, наборы инструкций включают поддержку конструкций высокоуровневого программирования.
 - Недостатки: нерегулярность потока команд, высокая стоимость аппаратной части, сложности с распараллеливанием вычислений.
 - RISC (Reduced Instruction Set Computer) — архитектура процессора с сокращённым набором команд.
 - В таких процессорах быстродействие увеличивается за счёт упрощения инструкций: их декодирование становится более простым, а время выполнения — меньшим.
 - Преимущества RISC-процессоров: повышение производительности обработки программ вычислительных задач, сокращение времени разработки благодаря использованию простых команд и минимума их форматов.
 - Недостатки RISC-процессоров: увеличивается семантический разрыв между исходным описанием и машинным кодом, сложность построения компилятор (поскольку программа

с языка высокого уровня должна транслироваться в микрокод с оптимизацией использования регистров.), высокие требования к быстродействию памяти.

- VLIW (Very Long Instruction Word) — архитектура процессоров с несколькими вычислительными устройствами. Характеризуется тем, что одна инструкция процессора содержит несколько операций, которые должны выполняться параллельно.
 - Основной принцип организации этой архитектуры сводится к тому, чтобы перенести нагрузку с периода исполнения в период компиляции. Эти задачи заранее решает компилятор, который располагает полной и достоверной информацией о регистрах процессора и генерирует оптимальный код, в котором нет конфликтов между регистрами.
 - Преимущества VLIW-процессоров: более тщательное планирование выполнения программы и оптимизация кода, меньше места тратится на функциональные блоки, отвечающие за управление (больше площади остаётся непосредственно на вычислительные ресурсы, такие как регистры, исполнительные устройства и кэш-память).
 - Недостатки VLIW-процессоров: сложный компилятор, возрастает время компиляции, сложно учесть динамику исполнения программы при наличии условных ветвлений на основе входящих динамических данных, программы для VLIW-процессоров могут быть гораздо длиннее, чем аналогичные программы для традиционных архитектур

54 Характеристики памяти



Характеристики памяти

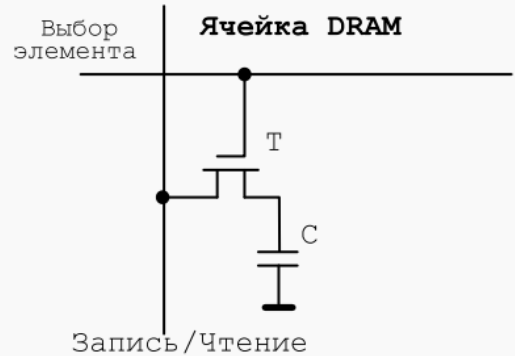
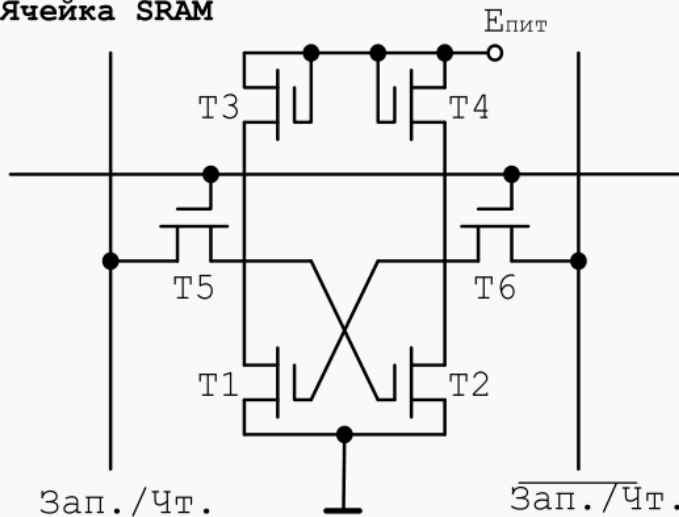
- Месторасположение
 - процессорные, внутренние, внешние
- Емкость
 - В метрических (Кило-) и двоичных (Киби-) множителях
- Единица пересылки
 - Слово, строка кэша, блок на диске
- Метод доступа
 - Произвольный (адресный), ориентированных на записи (прямой), последовательный, ассоциативный

- Быстродействие и временные соотношения
 - Время доступа T_d
 - Длительность цикла памяти (время обращения) $T_{\text{ц}}$
 - Время чтения и время записи
 - Время восстановления T_v
 - Скорость передачи информации
- Физический тип и особенности
- Стоимость



Статическая vs Динамическая

Ячейка SRAM



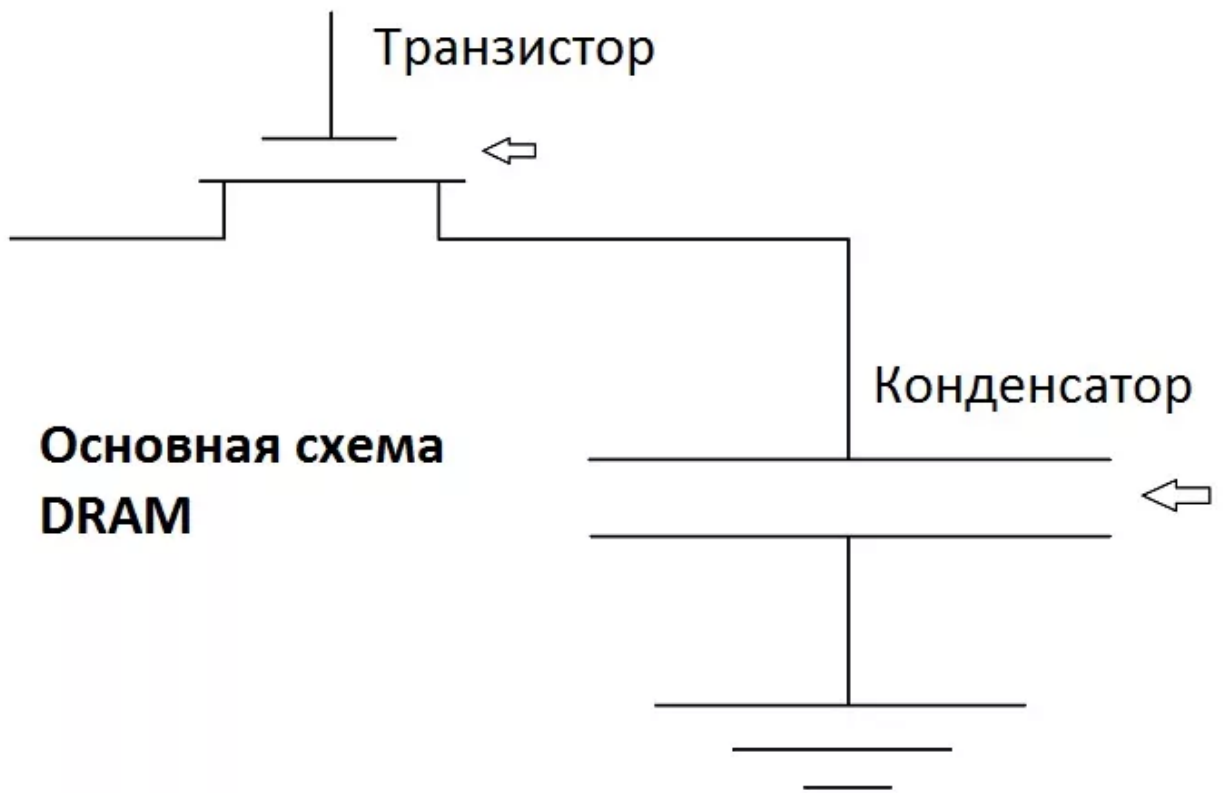
Разница между статической и динамической памятью заключается в режиме хранения информации. Статическая память способна хранить информацию сколько угодно долго при отсутствии обращений (но при наличии питающего напряжения). Ячейки такой памяти реализуются на триггерах — элементах с двумя устойчивыми состояниями. Динамическая память может работать только в динамическом режиме. При отсутствии обращения к её элементу со временем за счёт токов утечки конденсатор разряжается и информация теряется. Поэтому такая память требует постоянного периодического перезаряда конденсаторов.

Преимущества динамической памяти:

- низкая себестоимость
- высокая степень упаковки, позволяющая создавать чипы памяти большого объёма

Недостатки динамической памяти:

- относительно невысокое быстродействие, так как процесс зарядки и разрядки конденсатора занимает больше времени, чем переключение триггера
- высокая латентность, в основном из-за внутренней шины данных, в несколько раз более широкой, чем внешняя, и необходимости использования мультиплексора/демультиплексора
- необходимость регенерации заряда конденсатора, из-за его быстрого саморазряда, ввиду микроскопических размеров



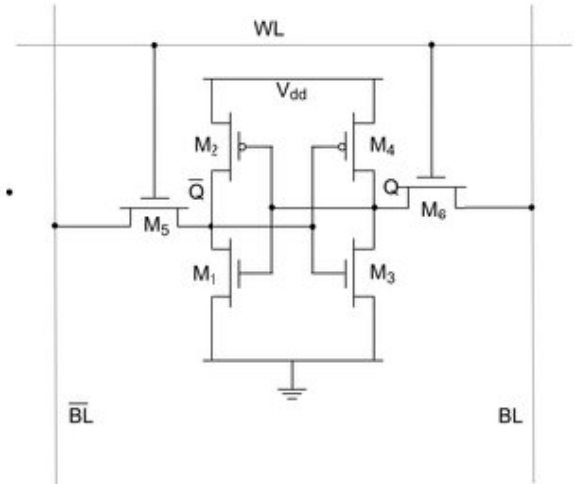
Статическая память

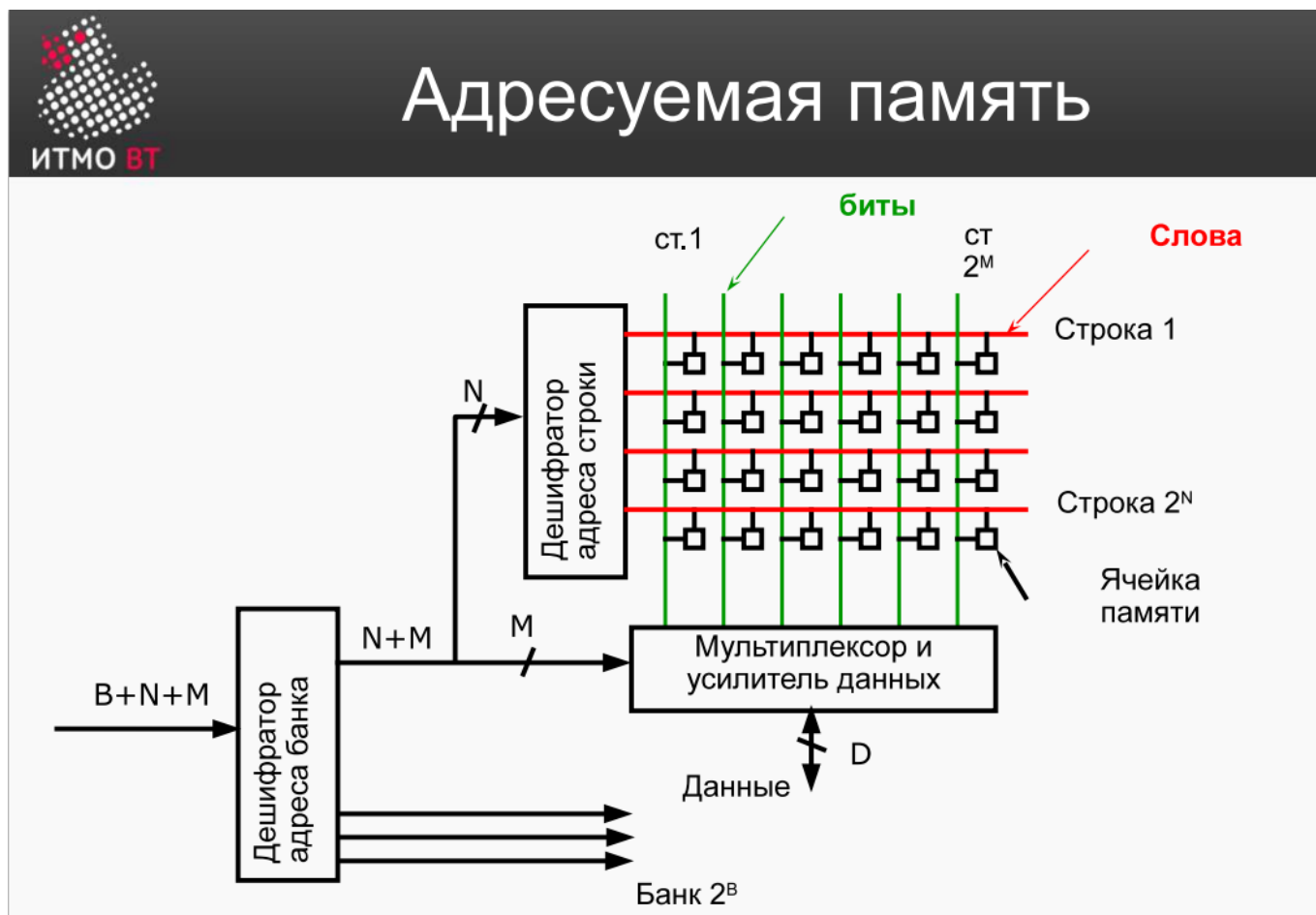
Ячейки памяти - электронные схемы *триггеры*.

В триггере хранится состояние (*включен/выключен*).

- **Преимущество:** высокое быстродействие.
- **Недостаток:** дороговизна.

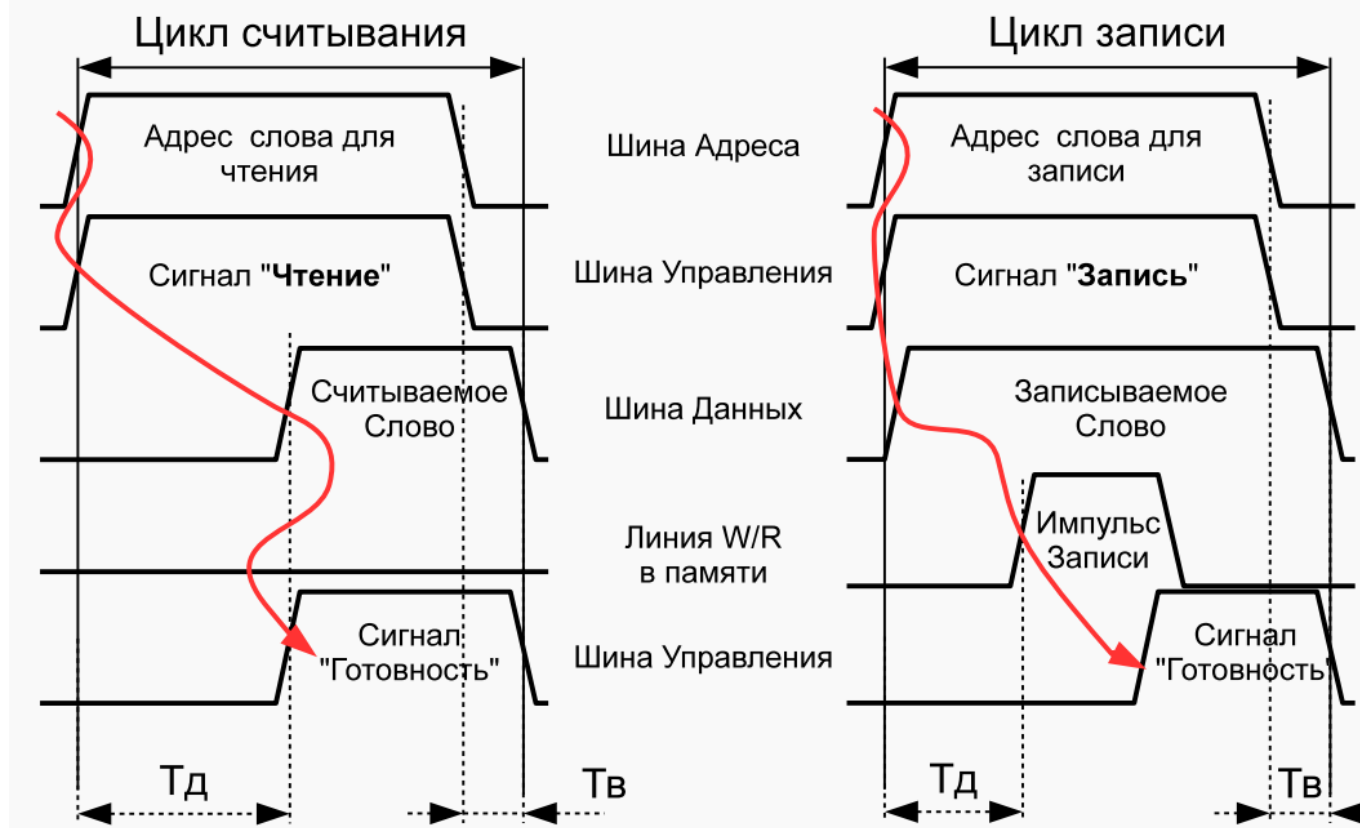
Микросхемы статической памяти используют в *кэш-памяти* для оптимизации работы процессора.



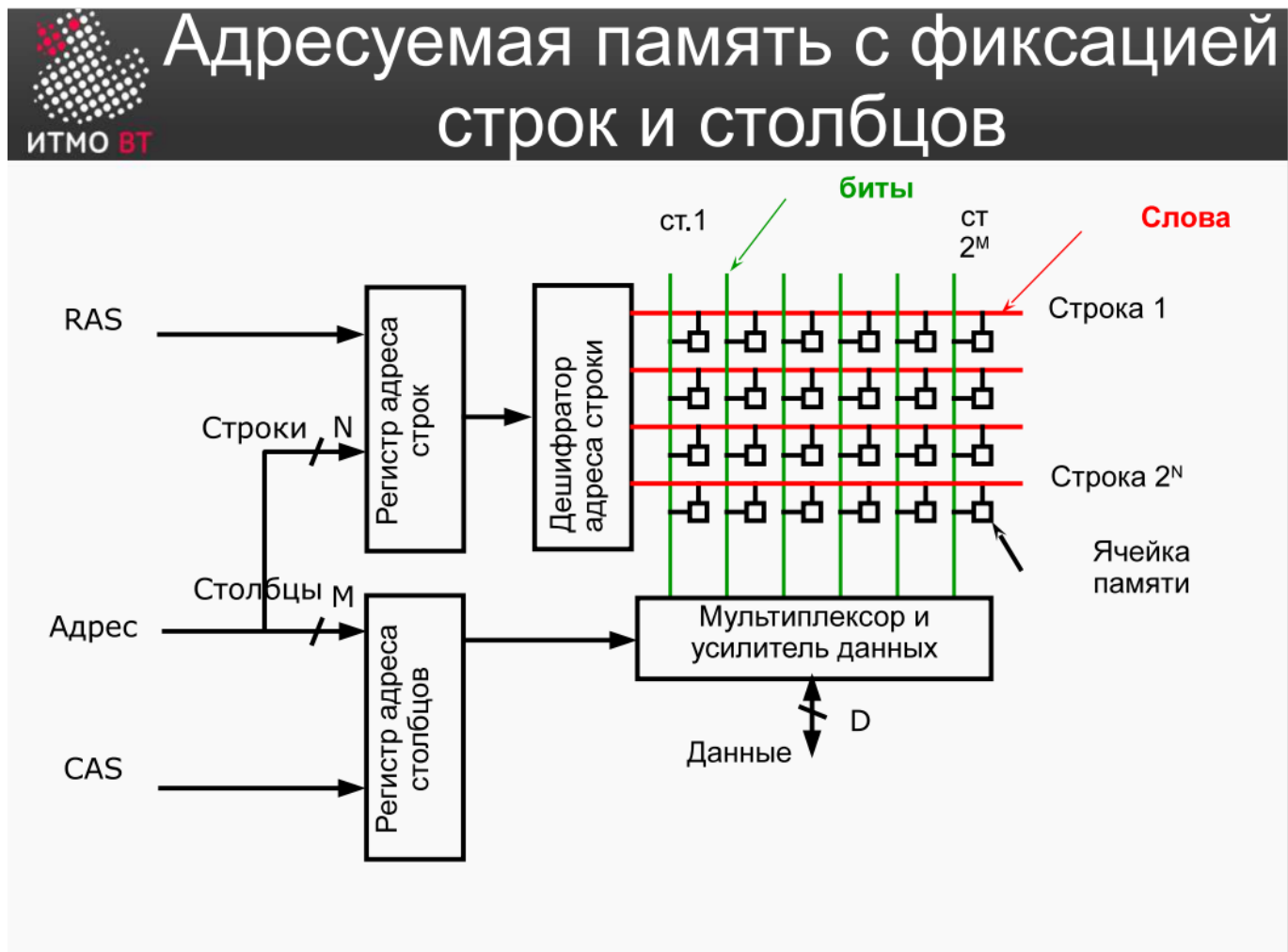


Адресное пространство — это набор адресов, может быть использован процессором для обращения к памяти. Каждой ячейке памяти соответствует определённый адрес. **Мультиплексор** - устройство, соединяющее несколько входов и один выход.

Оперативная память разбивается на чётное количество банков, которая состоит из страниц. Страница состоит из матриц, которые состоят из ячеек с 1 битом памяти

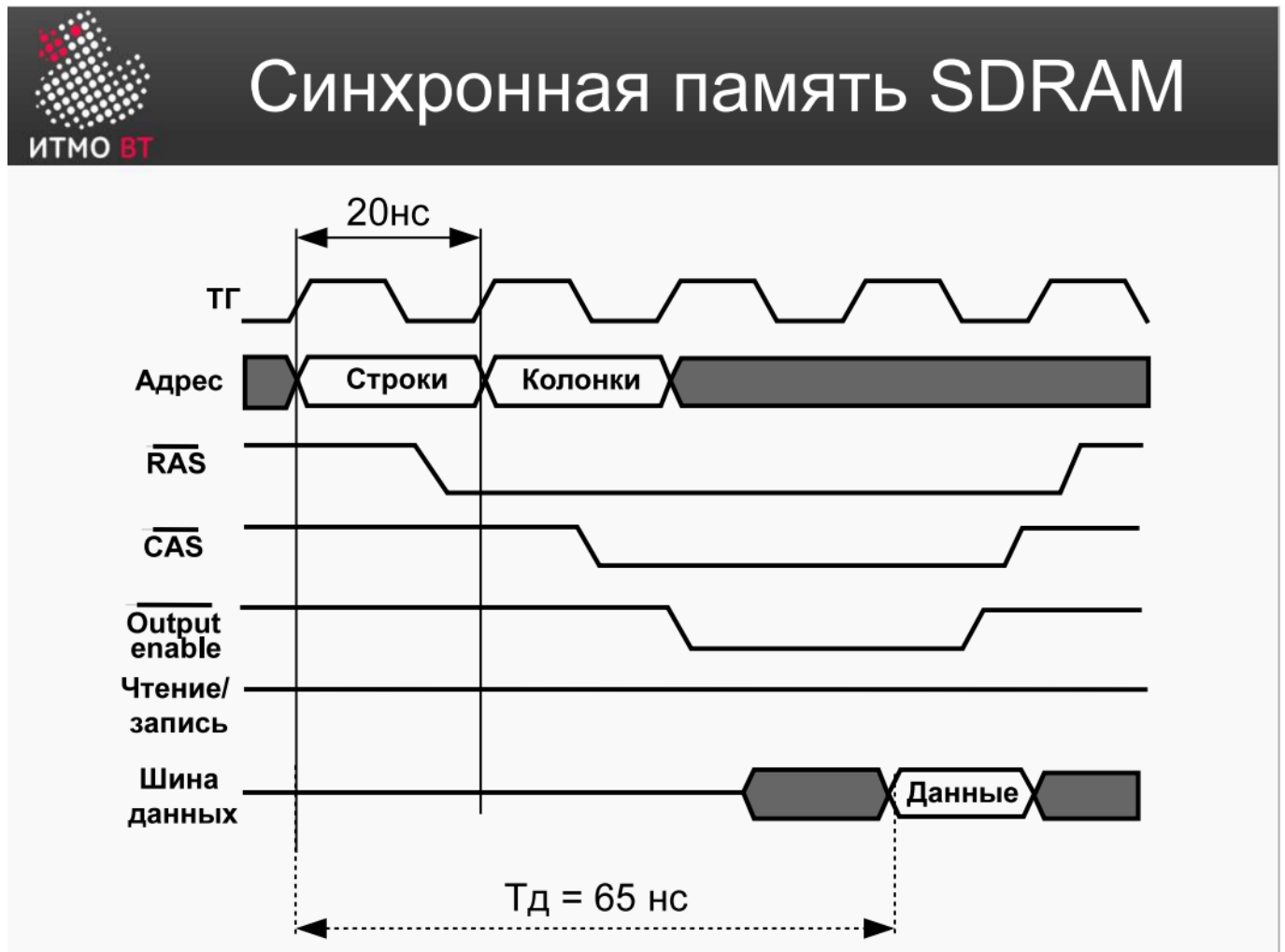


T_d - время доступа, временной интервал, когда процессор выставляет на адресной шине адрес ячейки памяти и после по шине приказов - приказ на чтение/запись, до момента операции связи адресуемой ячейки с шинной данных. T_v - время восстановления, необходим для приведения памяти в исходное состояние после того, как процессор снял с шин адреса сигналы, а также сигналы чтения/записи (и данные, при записи).



Доступ к ячейкам осуществляется с помощью указания адреса строки (RAS - ROW Address Strobe) и адреса столбца (CAS - Column Address Strobe)

1. На адресную шину подается номер строки, активируется сигнал выборки строки - RAS
2. Содержимое строки переносится в буферный усилитель данных
3. На адресную шину подается номера ячеек, сигнал - CAS, данные поступают на шину данных
4. Деактивируются сигналы RAS, CAS



Присутствует тактовый генератор Сигнала RAS и CAS инвертированы для удобства. За такт происходит 1 операция в памяти:

1. Передача адреса строки
2. Передача адреса столбца

59 Конструктивные особенности современной памяти

Burst mode - (пакетный режим) в нем проц. запрашивает данные из памяти не отдельными байтами, а виде пакетов (1 раз подали адрес, он запомнился в регистрах), состоящих из 32 или 64 бит, т.е. считывает 2 или 4 машинных слова за 1 такт. Обычно соответствуют строке кэша

Double Data Rate - передача данных как по фронту, так и по спаду, что увеличивает скорость в 2 раза

SPD - чип, содержащий идентификационную информацию, в котором производителем записывается информация о рабочих частотах и соответствующих задержках чипов памяти, необходимые для обеспечения нормальной работы модуля. Информация из SPD считывается BIOS на этапе самотестирования компьютера еще до загрузки операционной системы и позволяет автоматически оптимизировать параметры доступа к памяти.

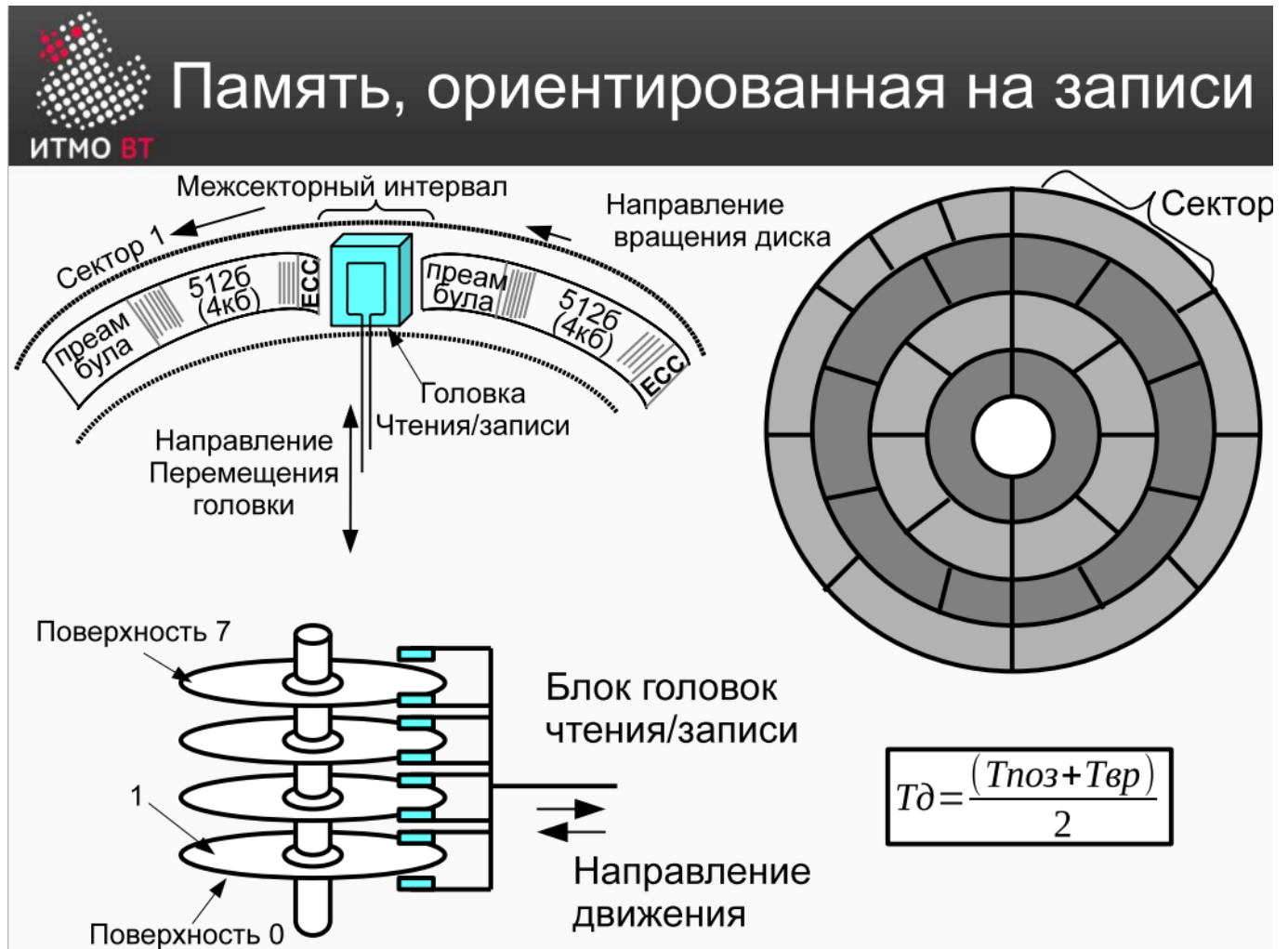
Interleaving - расслоение памяти, повышает производительность

DDR4-2133 8192MB PC4-17000:

- DDR - тип памяти Double Data Rate

- 4 - поколение
- 2133 - частота тактового генератора
- 8192MB - объём
- PC4-17000 - индекс производительности

60 Память, ориентированная на записи



Вспомогательные ЗУ обеспечивают большой объём памяти и относительно недороги, но обладают меньшим быстродействием, чем основная память микроЭВМ. Почти все такие ЗУ предназначены для длительного хранения программ и данных.

Дорожка - круговая последовательность битов. Каждая дорожка делится на секторы фиксированной длины. Один сектор содержит 512 байт данных. Перед данными - преамбула, которая позволяет головке синхронизироваться перед чтением/записи. После данных - код восстановления ошибок ЕСС, код Хэмминга. Между данными - межсекторный интервал.

$T_{\partial} = (T_{\text{поз}} + T_{\text{вр}})/2$ - время произвольного доступа - среднее время, за которое винчестер выполняет операцию позиционирования головки на произвольный сектор магнитного диска.

$T_{\text{поз}}$ - это время установки головки от центра диска до нужной дорожки.

$T_{\text{вр}}$ - это время поиска на дорожке нужного сектора.

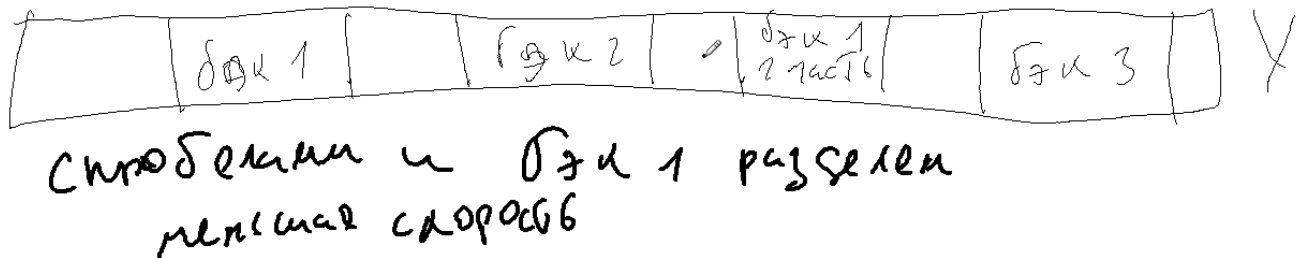
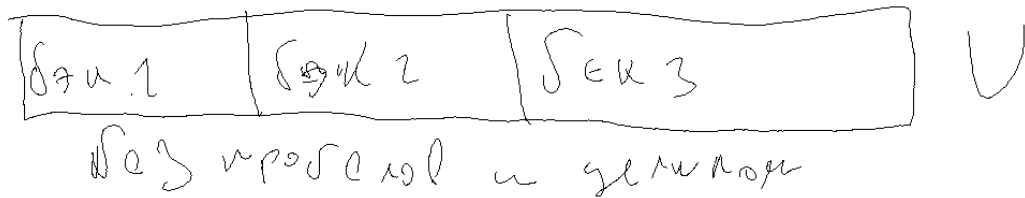
Винчестерные диски - накопители на жестких дисках с подвижной головкой, расположенные в герметизированном корпусе. Лучше защищены от пыли, что позволяет помещать головки намного ближе к поверхности диска, увеличивается уплотнение записи информации и ускорение доступа к ней. Обладают большим объёмом и сравнительно низкой стоимостью



На слайде ленточный накопитель. У него очень низкая стоимость хранения. Чем быстрее лента протягивается относительно головки, тем быстрее передаются память, для увеличения скорости люди решили, что будет двигаться не лента, а головка.

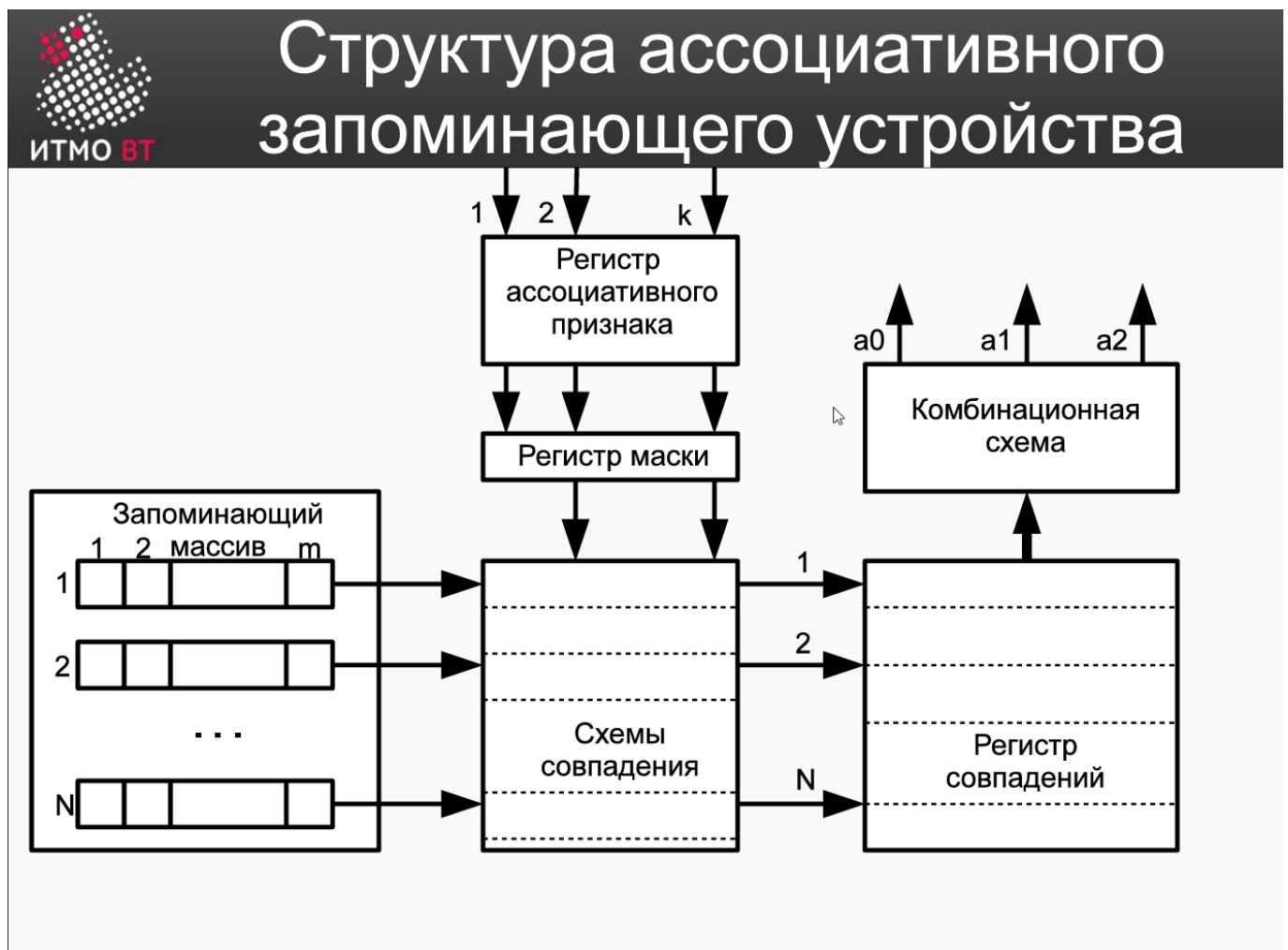
В современных лент. накопителях головка наклонена относительно ленты(как на слайде), поэтому на ленту все записывается под углом(низ слайда)

Время доступа очень большое, поэтому на ленте располагаются целиком “бэкапы” без промежутков (Эта его цитата, хер пойми что он пытался сказать, но я попробую дешифровать: Вот если я захочу получить на ленте доступ к определенному участку, мне придется ее перематывать до нужного места, это долго, поэтому использовать ее как жёсткий диск не круто. Ленты часто используются для хранения резервного копирования(по гейски бэкапы) и типо храним на ленте бэкапы без пропусков(пустых мест) и без какого либо разделения, поэтому перемотка происходит быстрее)



На прямую он не говорит, но все же как я понял: с произвольным доступом мы обращаемся к какой-то ячейке по адресу, имеем к ним прямой доступ, а вот при непрерывном доступе все находится в какой то одной последовательности(разделенной на блоки данных), и уже из этой последовательности мы нужные данные достаём путем перемотки, а не прямого обращения.

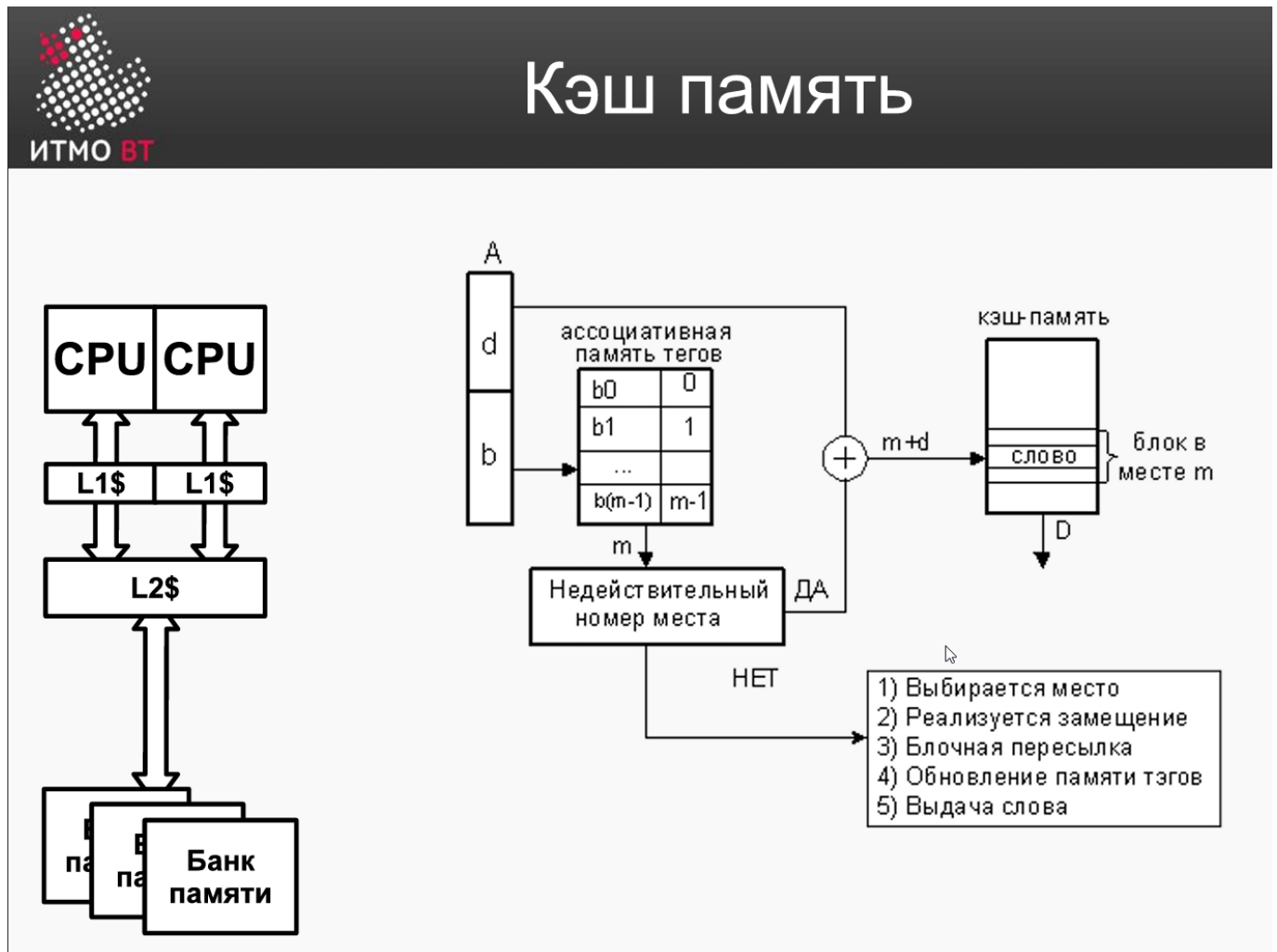
62 Структура ассоциативного запоминающего устройства



Ассоциативная память выбирает инфу не по адресу, а по нужному нам признаку

Есть регистр признака, а каждая ячейка памяти содержит схему сравнения(то есть видимо сравнивается с регистром признака), те ячейки, которые содержат нужную инфу передаются дальше. Подобным образом устроен Кэш

63 Кэш память



Адрес бьется на части, на тег и на просто адрес, и по тегу выбираются нужные ячейки

64 Пирамида памяти



Сверху - память которой мало и которая дорогая, ниже памяти становится больше и время доступа увеличивается

CPU - внутри процессора регистры, работают на тактовой частоте процессора если CPU не хватило можно перейти на кэш первого уровня

Нужно будет вызубрить эту пирамиду(объем и Тд особенно) Он будет требовать. X * столбец не надо учить. Значения конечно могут меняться(но нам все равно надо их выучить), но важно усвоить разницу между порядками(на сколько одна память быстрее/ больше другой, короче различие их) это тип поможет лучше оптимизировать программы

65 Влияние промахов кэш-памяти



Представьте, что процессор выполняет 1 команду за 1 наносекунду, а к памяти обращаемся за 100 наносекунд. Как влияет эффективность кэша на производительность?

99% - команд попадает в кэш - каждая за 1нс 1% - в память - каждая 100нс

При 100 командах - 199нс Если бы не было того 1% команд, то было бы в ДВА раза быстрее

Это и показывает график: например при 98 % попадания в кэш производительность будет 60% от полной

66 Сегментно-страничная виртуальная память

Страничная организация памяти **Принцип работы:** У нас есть виртуальное адресное пространство. Оно делится на фиксированные блоки, которые называются страницами. Физическая память делится на блоки такой же фиксированной длины - это называется страничными кадрами. Когда процессу нужно обратиться к памяти по определенному адресу в физической памяти, то адрес делится на два компонента - на номер страницы и смещение внутри страницы. По номеру страницы происходит поиск кадра в таблице страниц, а затем к нему прибавляется смещение для получения физического адреса. Отсутствует внешняя фрагментация (потому что память выделяется блоками фиксированного размера), но присутствует внутренняя фрагментация (из-за фиксированного размера страниц на них может оставаться пустое пространство)

Плюсы:

1. Простота управления памятью

2. Виртуальная память эффективно используется
3. Отсутствие внешней фрагментации

Минусы:

1. Внутренняя фрагментация
2. Увеличение времени доступа из-за того что надо доп преобразования делать

Сегментная реализация Принцип работы: Виртуальное адресное пространство делится на сегменты разной длины, и размер зависит от нужд памяти Виртуальный адрес состоит из номера сегмента, и смещения внутри сегмента В сегментных таблицах хранятся базовые адреса сегментов и их длина. А адрес виртуальной памяти получается из прибавления смещения и базового адреса сегмента

Плюсы:

1. Логичное разделение программы на сегменты
2. Отсутствие внутренней фрагментации и вследствие этого эффективное управление и использование памяти

Минусы:

1. Возможна внешняя фрагментация Усложнено управление памятью

Сегментно-страничная организация памяти - это гибрид между соответственно сегментной и страничной реализаций, который взял лучшее из обеих концепций. Принцип работы: Память делится на сегменты, и каждый сегмент - на страницы

Вывод:

1. Сегменты - логические единицы программы с переменным размером
2. Страницы - фиксированные блоки внутри каждого сегмента Адресация происходит по следующему принципу: Номер сегмента -> номер страницы -> смещение

Плюсы:

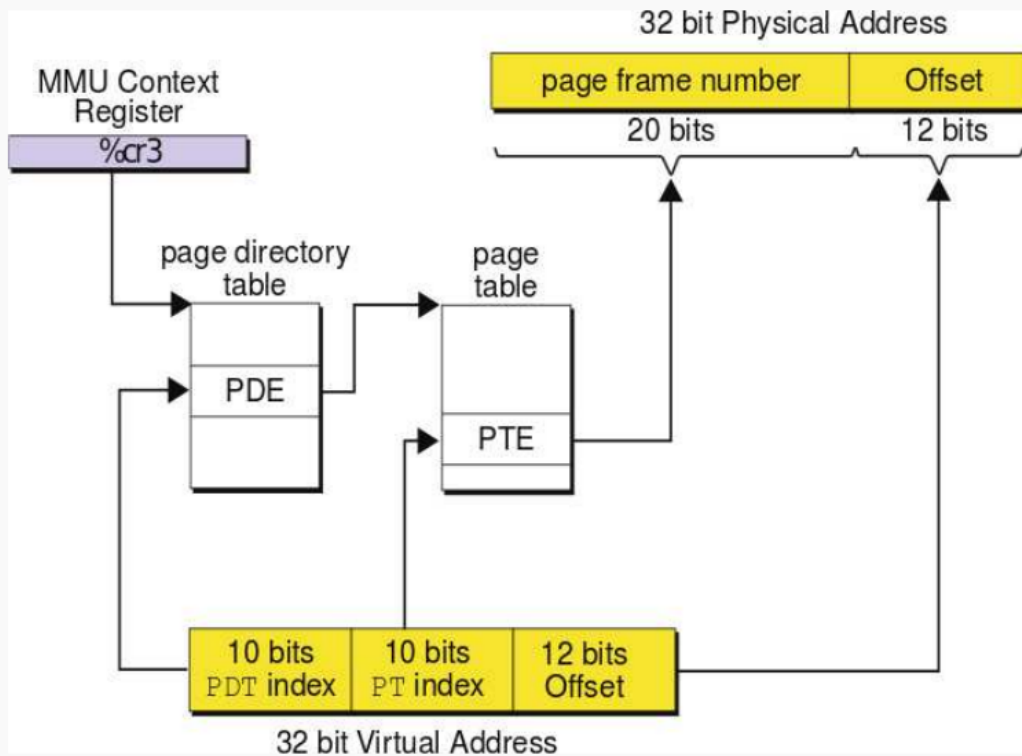
1. Минимальная фрагментация
2. Гибкость и защита (можно раздавать права доступа чему угодно)
3. Поддержка больших адресных пространств
4. Эффективное управление памятью

Минусы:

1. Сложность управления памятью (еще сильнее чем у сегментной)
2. Увеличение накладных расходов
3. Очень много обращений к таблицам



Архитектура x86: MMU



MMU - блок управления памятью - Memory Management Unit - компонент, отвечающий за управление доступом к памяти. На входе поступает виртуальный адрес, в таблице преобразований смотрит наличие страницы виртуальной памяти и физический адрес.

TBL - Кэширует часто используемые преобразования • Чаще всего отдельный для адреса и памяти • Организован в виде ассоциативной памяти • Реализуется через кэши первого-четвертого уровня

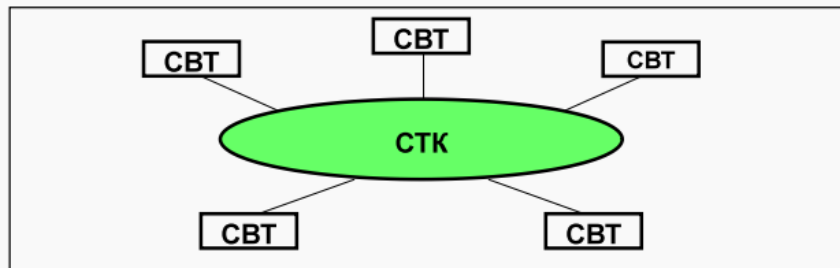


История сети Internet

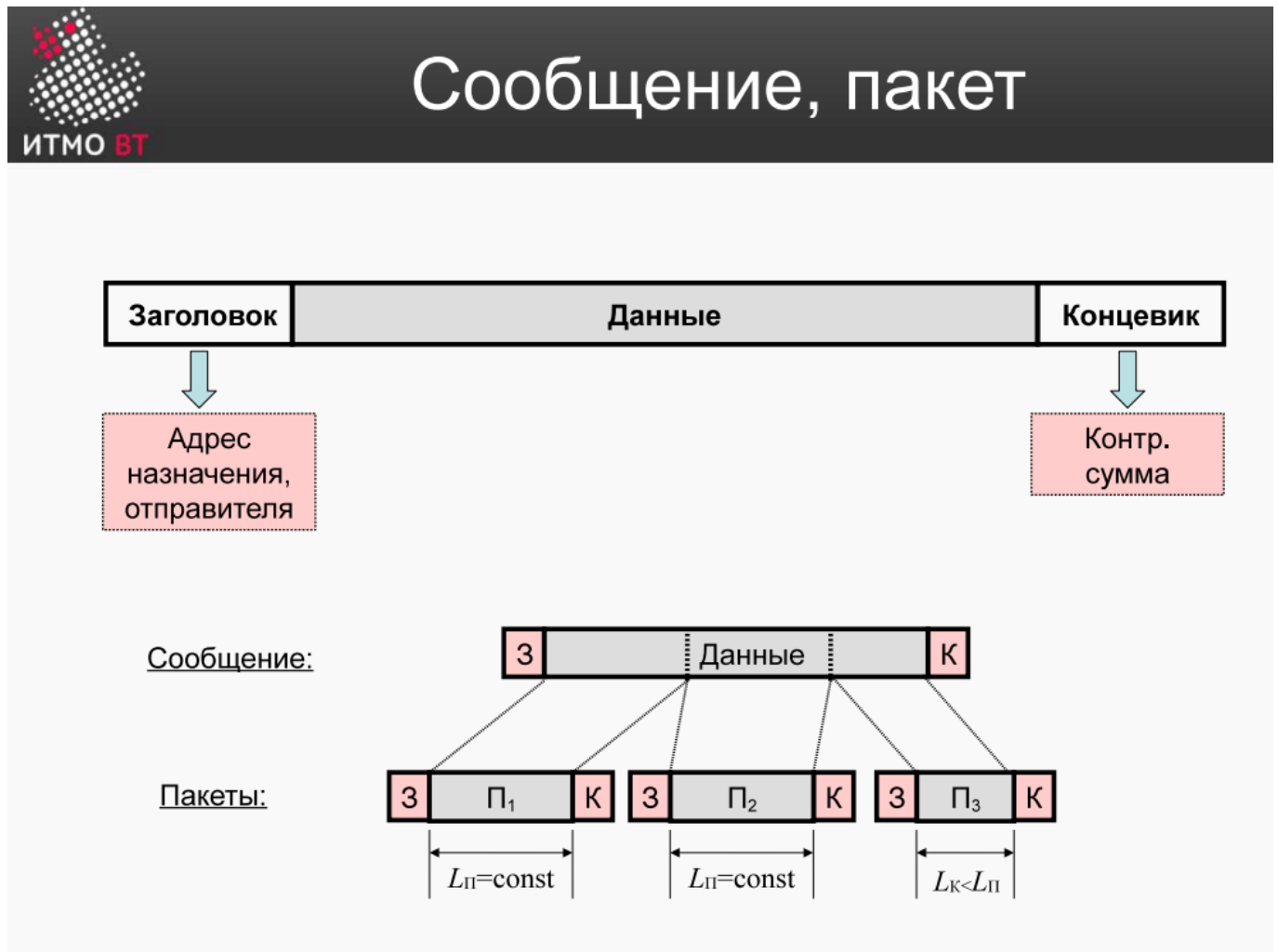
- 1957 — запуск первого спутника земли
- 1958 — Advanced Research Projects Agency (ARPA).
- 1963 — J.C.R.Licklider — первая концепция компьютерной сети
- 1969 — ARPANET в ведущих лабораториях и исследовательских центра США
- 1976 — Xerox — локальная сеть Ethernet
- 1982 — ARPA — единый стек протоколов TCP/IP
- 1983-84 — FidoNet и BBS, ARPANET → Internet
- 1991 - Tim Berners-Lee, CERN, концепция WWW, первый http сервер



Понятие сети ЭВМ



- Средства вычислительной техники (СВТ): ЭВМ, вычислительные комплексы (ВК) и вычислительные системы (ВС) - реализуют обработку данных.
- Средства телекоммуникаций (связи) (СТК): совокупность каналов связи и каналообразующей аппаратуры - реализуют передачу данных.
- Сеть ЭВМ (вычислительная сеть, компьютерная сеть) = СВТ+СТК



Сообщение

- Логически завершенная порция данных, например, запрос на передачу файла, ответ на этот запрос, содержащий весь файл и т.д. Сообщения могут иметь произвольную длину, от нескольких байт до многих мегабайт.
- Цифровые данные определенного формата, предназначенные для передачи
- Длинные сообщения бьются на пакеты (обычно по 1500 байт)
- Каждое сообщение и пакет содержат заголовок

Пакет — это сравнительно небольшая часть сообщения, которая транспортируется по сети как независимый информационный блок. Пакеты обычно имеют длину в 1500 байт. Каждый пакет снабжается заголовком, в котором указывается адресная информация, необходимая для доставки пакета на узел назначения, а также номер пакета, который будет использоваться узлом назначения для сборки сообщения.

71 Модель взаимодействия открытых систем (OSI)

Модель взаимодействия открытых систем (OSI, Open Systems Interconnection) — это модель для стандартизации процессов взаимодействия компьютерных систем в сети. Модель OSI состоит из семи уровней, каждый из которых выполняет определенные функции и служит основой для передачи данных от одного устройства к другому.

- 1. Физический уровень (Physical Layer): Отвечает за передачу необработанных битов по физическим средам (кабели, радиоволны и т.д.)
- Канальный уровень (Data Link Layer):
 - Обеспечивает передачу данных между непосредственно связанными узлами
 - Делится на два подуровня:
 - * Logical Link Control (LLC) — управление логическими соединениями.
 - * Media Access Control (MAC) — управление доступом к физической среде.
- Сетевой уровень (Network Layer):
 - Отвечает за маршрутизацию пакетов данных между различными сетями.
 - Определяет логические адреса (например, IP-адреса) и управляет фрагментацией пакетов.
- Транспортный уровень (Transport Layer): Обеспечивает надежную передачу данных между конечными узлами.
- Сессионный уровень (Session Layer): Устанавливает, управляет и завершает сеансы связи между приложениями.
- Представительский уровень (Presentation Layer): Отвечает за преобразование данных в формат, понятный приложению.
- Прикладной уровень (Application Layer): Предоставляет сетевые услуги конечным пользователям и приложениям.

Модель OSI помогает разработчикам и инженерам понимать и проектировать сетевые протоколы и системы. Она служит основой для создания стандартов и протоколов, которые обеспечивают совместимость и взаимодействие различных систем и устройств в сети.

72 Модель TCP/IP

Модель TCP/IP (Transmission Control Protocol/Internet Protocol) — это набор сетевых протоколов, который используется для связи в интернете и других сетях. Модель TCP/IP состоит из четырех уровней, которые соответствуют различным функциям передачи данных:

- 1. Уровень сетевого интерфейса (Network Interface Layer): Этот уровень отвечает за физическую передачу данных по сети и включает в себя все технологии, используемые для передачи данных между устройствами.
- Интернет-уровень (Internet Layer): Основная задача этого уровня — обеспечить маршрутизацию пакетов данных между различными сетями.
- Транспортный уровень (Transport Layer): Этот уровень отвечает за надежную передачу данных между конечными узлами.
- Прикладной уровень (Application Layer): Этот уровень предоставляет интерфейсы для приложений и пользователей для доступа к сетевым услугам.

Сравнение с моделью OSI

Хотя модель TCP/IP и модель OSI обе описывают процессы передачи данных в сети, они имеют различия в количестве уровней и в их функциональности. Модель OSI состоит из семи уровней, в то время как модель TCP/IP имеет четыре уровня. В TCP/IP некоторые функции из модели OSI объединены в один уровень.

73 Уровень передающей среды

Уровень передающей среды (или уровень физической среды) — это самый низкий уровень в модели ТСП/IP, который отвечает за физическую передачу данных по сети.

Основные функции уровня передающей среды:

- Физическая передача данных: Определяет, как данные преобразуются в электрические, оптические или радиосигналы для передачи по физическим носителям (например, кабелям, оптоволокну или беспроводным каналам)
- Типы сетевых технологий:
 - Охватывает различные технологии передачи данных, такие как Ethernet, Wi-Fi, Bluetooth и другие
 - Устанавливает стандарты для различных физических интерфейсов и соединений
- Управление доступом к среде: Определяет, как устройства получают доступ к общей среде передачи и как предотвращаются коллизии (например, с помощью протоколов CSMA/CD для Ethernet)
- Обработка ошибок: В некоторых случаях уровень передающей среды может обеспечивать базовые механизмы обнаружения ошибок, такие как контрольные суммы.
- Сигнализация: Определяет параметры сигналов, такие как уровень напряжения, частота и модуляция, которые используются для передачи данных.

Примеры технологий на уровне передающей среды:

- Ethernet
- Wi-Fi
- Bluetooth

Уровень передающей среды является критически важным для обеспечения физической связи между устройствами в сети. Он определяет, как данные передаются и принимаются, а также какие технологии используются для этого.

74 Канальный уровень Ethernet

Канальный уровень Ethernet относится к второму уровню модели OSI, который отвечает за передачу данных между узлами в локальной сети. Этот уровень включает в себя как управление доступом к среде передачи, так и форматирование данных для передачи.

Вот основные аспекты канального уровня Ethernet:

- Формат кадра: Данные, передаваемые по сети Ethernet, упаковываются в кадры. Каждый кадр содержит заголовок и контрольную сумму, а также саму полезную нагрузку. Заголовок включает адреса отправителя и получателя (MAC-адреса), а контрольная сумма (CRC) используется для проверки целостности данных.
- MAC-адресация: Каждый сетевой интерфейс имеет уникальный MAC-адрес, который используется для идентификации устройства в локальной сети.
- Методы доступа к среде: Ethernet использует метод CSMA/CD (Carrier Sense Multiple Access with Collision Detection) для управления доступом к среде передачи. Это означает, что устройства сначала "слушают" среду, чтобы убедиться, что она свободна, прежде чем начать передачу данных. Если происходит коллизия (одновременно передают несколько устройств), они останавливаются и повторяют попытку через случайный интервал времени.

- Типы Ethernet: Существуют различные стандарты Ethernet, которые различаются по скорости передачи и типу используемых кабелей.
- Сетевые устройства: На канальном уровне работают такие устройства, как коммутаторы и мосты, которые помогают направлять кадры между устройствами в сети, основываясь на их MAC-адресах.

Канальный уровень Ethernet играет ключевую роль в обеспечении надежной и эффективной передачи данных в локальных сетях.

75 Сетевой уровень IP

Сетевой уровень, представленный протоколом IP (Internet Protocol), является третьим уровнем модели OSI и отвечает за маршрутизацию данных между узлами в различных сетях. Основные функции и характеристики сетевого уровня IP включают:

- Маршрутизация: Сетевой уровень отвечает за определение маршрута, по которому данные должны следовать от источника к получателю.
- IP-адресация: Каждый узел в сети имеет уникальный IP-адрес, который используется для его идентификации. IP-адреса бывают двух типов:
 - IPv4: Использует 32-битные адреса (например, 192.168.1.1)
 - IPv6: Использует 128-битные адреса (например, 2001:0db8:85a3:0000:0000:8a2e:0370:7334)
- Фрагментация и сборка: Если размер пакета данных превышает максимальный размер передачи на каком-либо из маршрутов, протокол IP может разбивать пакет на более мелкие фрагменты для передачи.
- Протоколы управления: На сетевом уровне также работают протоколы, которые используются для передачи сообщений об ошибках и диагностики сетевых проблем
- Ненадежная доставка: Протокол IP не гарантирует надежную доставку пакетов. Это означает, что пакеты могут быть потеряны, дублированы или доставлены в неправильном порядке.
- Подсети: IP-адреса могут быть разделены на подсети, что позволяет организовать адресное пространство более эффективно и управлять трафиком в больших сетях.

Сетевой уровень IP является основой для функционирования интернета и многих локальных сетей, обеспечивая маршрутизацию и адресацию для передачи данных между различными устройствами.

76 Сетевой уровень IP: маршрутизация

Функции: Сетевой уровень отвечает за маршрутизацию пакетов данных от источника к получателю, возможно через несколько промежуточных узлов. Основной протокол - IP (Internet Protocol), который предоставляет логические адреса и определяет пути передачи данных между узлами.

Маршрутизация: Процесс выбора оптимального пути в сети для передачи пакетов. Включает использование маршрутизаторов и таблиц маршрутов. Алгоритмы маршрутизации могут быть статическими (ручное конфигурирование маршрутов) или динамическими (автоматическое обновление маршрутов на основе текущих условий в сети).

77 DHCP

Функции: DHCP автоматизирует назначение IP-адресов и других параметров сетевой конфигурации (включая маску подсети, шлюз по умолчанию и DNS-серверы) устройствам в сети. Это упрощает управление сетью, особенно при большом количестве устройств.

Процесс: Устройства отправляют DHCP DISCOVER запрос для нахождения DHCP-сервера. Сервер отвечает предложениями (DHCP OFFERы), и устройство выбирает один из них, отправляя DHCP REQUEST. Сервер подтверждает назначение параметров с помощью DHCP ACK.

78 Сервис имен DNS и другие

DNS: DNS преобразует понятные человеку доменные имена (например, example.com) в IP-адреса, которые использует сеть. Это распределённая система серверов, обеспечивающая такую трансляцию.

Другие сервисы имен: В локальных или специфичных сетевых средах могут использоваться другие системы разрешения имен, такие как WINS (Windows Internet Name Service) и mDNS (Multicast DNS), обеспечивающие аналогичные функции, но с более узким фокусом.

79 Транспортный уровень

Функции: Транспортный уровень обеспечивает надёжную передачу данных между узлами в сети. Это включает контроль передачи, установление соединений и обеспечение целостности данных.

Протоколы: Основными протоколами являются TCP (Transmission Control Protocol) и UDP (User Datagram Protocol). TCP обеспечивает надёжную и упорядоченную доставку данных с подтверждением получения, а UDP предлагает более быстрый, но ненадёжный способ передачи без установления соединения и подтверждения.

80 Прикладной уровень

Функции: Прикладной уровень предоставляет интерфейсы и протоколы, через которые приложения взаимодействуют с сетью. Этот уровень отвечает за непосредственное обеспечение сетевых услуг для пользователей и приложений.

Протоколы и приложения: Включает такие протоколы как HTTP (для передачи веб-контента), SMTP (для отправки электронной почты), FTP (для передачи файлов), и множество других, которые обеспечивают конкретные прикладные функции и услуги.

Все эти уровни и сервисы работают вместе в рамках модели OSI (Open Systems Interconnection) или модели TCP/IP, обеспечивая комплексные сетевые функции и услуги для пользователей и устройств в сети.